

13. 스레드 풀과 Executor 프레임워크2

#1.인강/0.자바/5.자바-고급1편

- /ExecutorService 우아한 종료 - 소개
- /ExecutorService 우아한 종료 - 구현
- /Executor 스레드 풀 관리 - 코드
- /Executor 스레드 풀 관리 - 분석
- /Executor 전략 - 고정 풀 전략
- /Executor 전략 - 캐시 풀 전략
- /Executor 전략 - 사용자 정의 풀 전략
- /Executor 예외 정책
- /정리

ExecutorService 우아한 종료 - 소개

고객의 주문을 처리하는 서버를 운영중이라고 가정하자.

만약 서버 기능을 업데이트를 위해서 서버를 재시작해야 한다고 가정해보자.

이때 서버 애플리케이션이 고객의 주문을 처리하고 있는 도중에 갑자기 재시작 된다면, 해당 고객의 주문이 제대로 진행 되지 못할 것이다.

가장 이상적인 방향은 새로운 주문 요청은 막고, 이미 진행중인 주문은 모두 완료한 다음에 서버를 재시작 하는 것이 가장 좋을 것이다.

이처럼 서비스를 안정적으로 종료하는 것도 매우 중요하다.

이렇게 문제 없이 우아하게 종료하는 방식을 **graceful shutdown**이라 한다.

이런 관점에서 `ExecutorService`의 종료에 대해서 알아보자.

ExecutorService의 종료 메서드

`ExecutorService`를 줄여서 앞으로 서비스라고 하겠다.

`ExecutorService`에는 종료와 관련된 다양한 메서드가 존재한다.

서비스 종료

- `void shutdown()`
 - 새로운 작업을 받지 않고, 이미 제출된 작업을 모두 완료한 후에 종료한다.
 - 논 블로킹 메서드(이 메서드를 호출한 스레드는 대기하지 않고 즉시 다음 코드를 호출한다.)

- `List<Runnable> shutdownNow()`
 - 실행 중인 작업을 중단하고, 대기 중인 작업을 반환하며 즉시 종료한다.
 - 실행 중인 작업을 중단하기 위해 인터럽트를 발생시킨다.
 - 논 블로킹 메서드

서비스 상태 확인

- `boolean isShutdown()`
 - 서비스가 종료되었는지 확인한다.
- `boolean isTerminated()`
 - `shutdown()`, `shutdownNow()` 호출 후, 모든 작업이 완료되었는지 확인한다.

작업 완료 대기

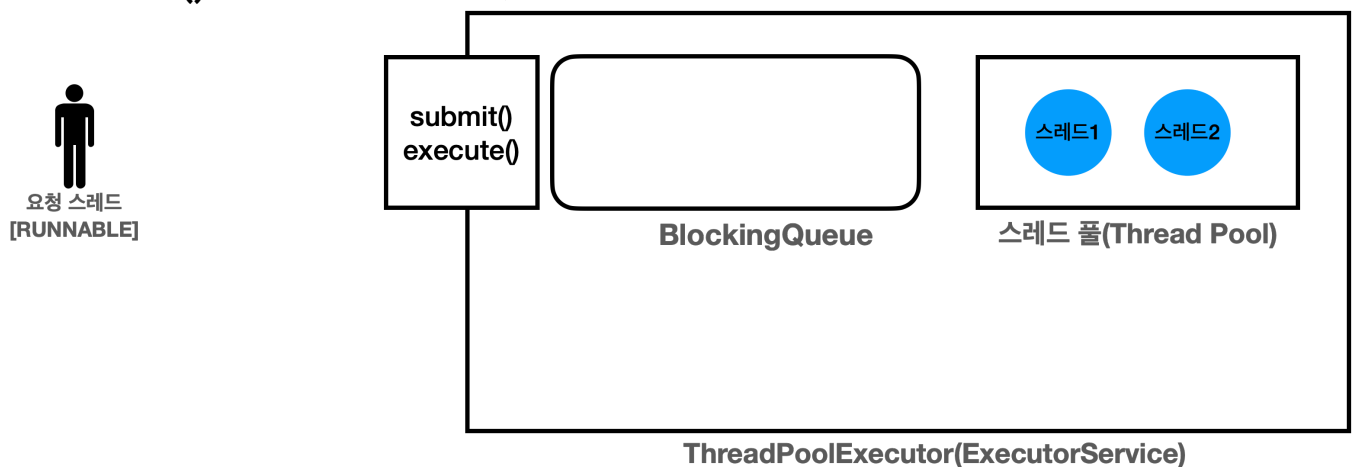
- `boolean awaitTermination(long timeout, TimeUnit unit) throws InterruptedException`
 - 서비스 종료시 모든 작업이 완료될 때까지 대기한다. 이때 지정된 시간까지만 대기한다.
 - 블로킹 메서드

close()

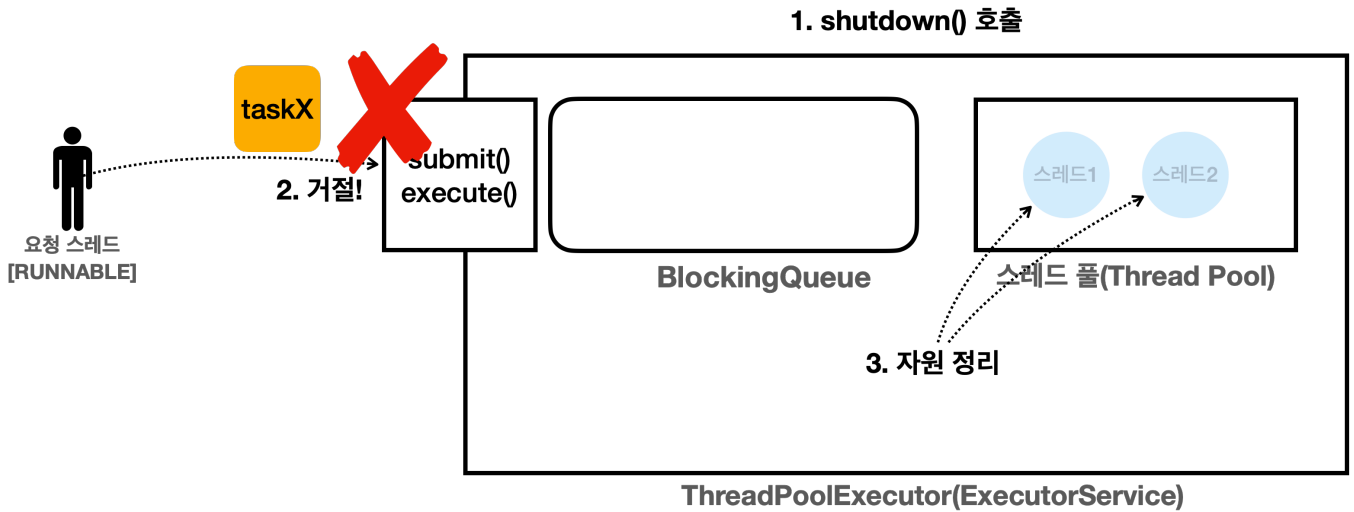
`close()` 는 자바 19부터 지원하는 서비스 종료 메서드이다. 이 메서드는 `shutdown()` 과 같다고 생각하면 된다.

- 더 정확히는 `shutdown()` 을 호출하고, 하루를 기다려도 작업이 완료되지 않으면 `shutdownNow()` 를 호출한다.
- 호출한 스레드에 인터럽트가 발생해도 `shutdownNow()` 를 호출한다.

shutdown() - 처리중인 작업이 없는 경우

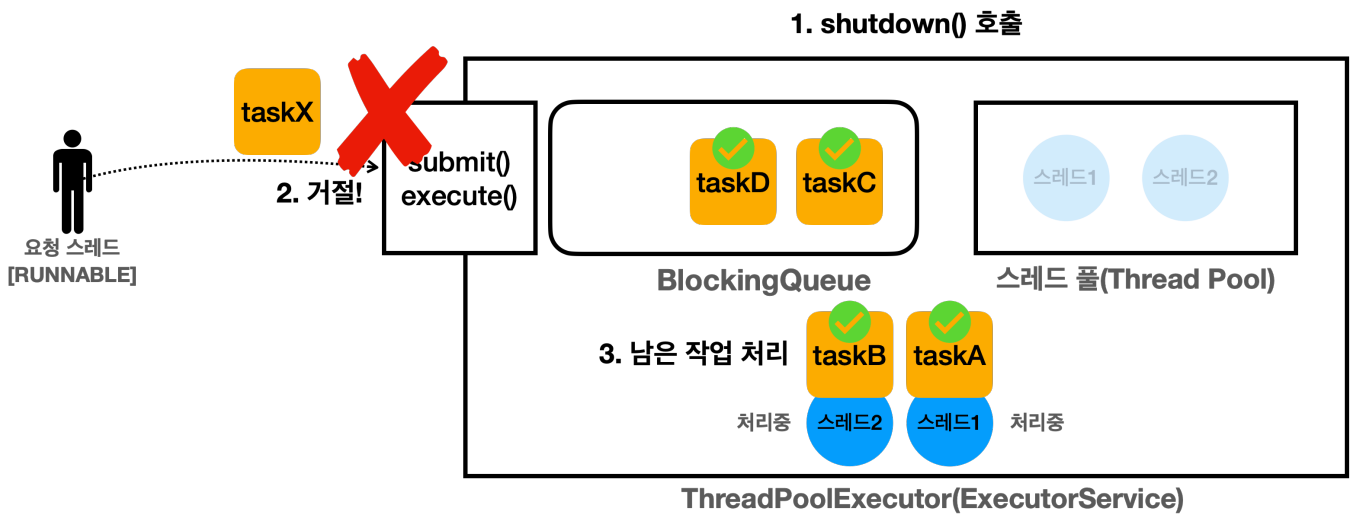


- `ExecutorService` 에 아무런 작업이 없고, 스레드만 2개 대기하고 있다.

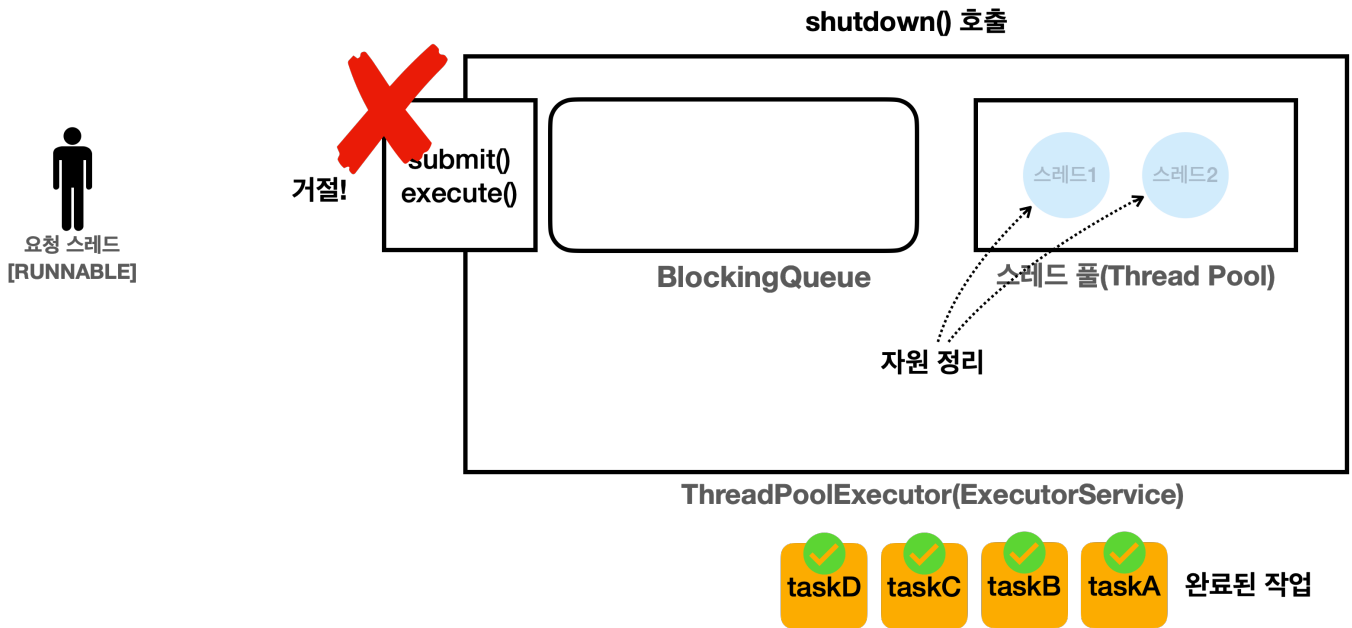


- shutdown() 을 호출한다.
- ExecutorService 는 새로운 요청을 거절한다.
 - 거절시 기본적으로 `java.util.concurrent.RejectedExecutionException` 예외가 발생한다.
- 스레드 풀의 자원을 정리한다.

shutdown() - 처리중인 작업이 있는 경우

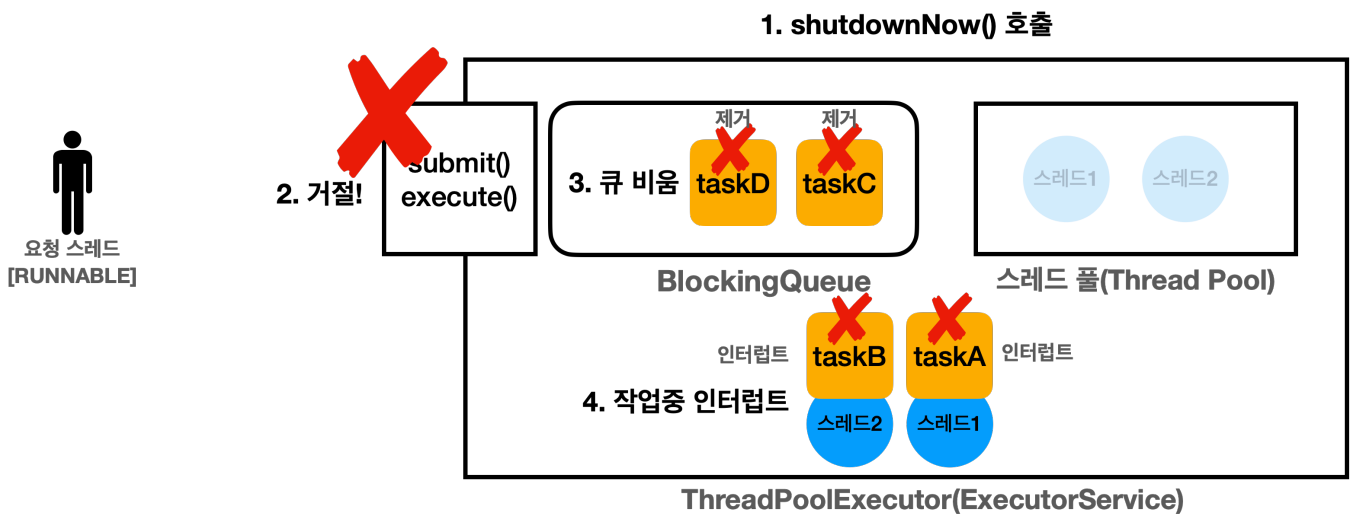


- shutdown() 을 호출한다.
- ExecutorService 는 새로운 요청을 거절한다.
- 스레드 풀의 스레드는 처리중인 작업을 완료한다.
- 스레드 풀의 스레드는 큐에 남아있는 작업도 모두 꺼내서 완료한다.



- 모든 작업을 완료하면 자원을 정리한다.
- 결과적으로 처리중이던 taskA, taskB는 물론이고, 큐에 대기중이던 taskC, taskD도 완료된다.

shutdownNow() - 처리중인 작업이 있는 경우



- shutdownNow() 를 호출한다.
- ExecutorService 는 새로운 요청을 거절한다.
- 큐를 비우면서, 큐에 있는 작업을 모두 꺼내서 컬렉션으로 반환한다.
 - `List<Runnable> runnables = es.shutdownNow()`
- 작업 중인 스레드에 인터럽트가 발생한다.
 - 작업 중인 taskA, taskB 는 인터럽트가 걸린다.
 - 큐에 대기중인 taskC, taskD 는 수행되지 않는다.
- 작업을 완료하면 자원을 정리한다.

ExecutorService 우아한 종료 - 구현

`shutdown()` 을 호출해서 이미 들어온 모든 작업을 다 처리하고 서비스를 우아하게 종료(graceful shutdown)하는 것이 가장 이상적이지만, 갑자기 요청이 너무 많이 들어와서 큐에 대기중인 작업이 너무 많아 작업 완료 어렵거나, 작업이 너무 오래 걸리거나, 또는 버그가 발생해서 특정 작업이 끝나지 않을 수 있다. 이렇게 되면 서비스가 너무 늦게 종료되거나, 종료되지 않는 문제가 발생할 수 있다.

이럴 때는 보통 우아하게 종료하는 시간을 정한다. 예를 들어서 60초까지는 작업을 다 처리할 수 있게 기다리는 것이다. 그리고 60초가 지나면, 무언가 문제가 있다고 가정하고 `shutdownNow()` 를 호출해서 작업들을 강제로 종료한다.

close()

`close()` 의 경우 이렇게 구현되어 있다. `shutdown()` 을 호출하고, 하루를 기다려도 작업이 완료되지 않으면 `shutdownNow()` 를 호출한다. 그런데 대부분 하루를 기다릴 수는 없을 것이다.

방금 설명한대로 우선은 `shutdown()` 을 통해 우아한 종료를 시도하고, 10초간 종료되지 않으면 `shutdownNow()` 통해 강제 종료하는 방식을 구현해보자. (학습용 예제에서 60초는 너무 길다.)

참고로 구현할 `shutdownAndAwaitTermination()` 은 `ExecutorService` 공식 API 문서에서 제안하는 방식이다.

```
package thread.executor;

import java.util.concurrent.*;

import static thread.executor.ExecutorUtils.printState;
import static util.MyLogger.log;

public class ExecutorShutdownMain {

    public static void main(String[] args) throws InterruptedException {
        ExecutorService es = Executors.newFixedThreadPool(2);
        es.execute(new RunnableTask("taskA"));
        es.execute(new RunnableTask("taskB"));
        es.execute(new RunnableTask("taskC"));
        es.execute(new RunnableTask("longTask", 100_000)); // 100초 대기
        printState(es);
        log("== shutdown 시작 ==");
        shutdownAndAwaitTermination(es);
    }
}
```

```

        log("== shutdown 완료 ==");
        printState(es);
    }

    static void shutdownAndAwaitTermination(ExecutorService es) {
        es.shutdown(); // non-blocking, 새로운 작업을 받지 않는다. 처리 중이거나, 큐에 이
        미 대기중인 작업은 처리한다. 이후에 풀의 스레드를 종료한다.
        try {
            // 이미 대기중인 작업들을 모두 완료할 때 까지 10초 기다린다.
            log("서비스 정상 종료 시도");
            if (!es.awaitTermination(10, TimeUnit.SECONDS)) {
                // 정상 종료가 너무 오래 걸리면...
                log("서비스 정상 종료 실패 -> 강제 종료 시도");
                es.shutdownNow();
                // 작업이 취소될 때 까지 대기한다.
                if (!es.awaitTermination(10, TimeUnit.SECONDS)) {
                    log("서비스가 종료되지 않았습니다.");
                }
            }
        } catch (InterruptedException ex) {
            // awaitTermination()으로 대기중인 현재 스레드가 인터럽트 될 수 있다.
            es.shutdownNow();
        }
    }
}

```

실행 결과

```

16:58:30.966 [      main] [pool=2, active=2, queuedTasks=2, completedTasks=0]
16:58:30.966 [pool-1-thread-1] taskA 시작
16:58:30.966 [pool-1-thread-2] taskB 시작
16:58:30.968 [      main] == shutdown 시작 ==
16:58:30.969 [      main] 서비스 정상 종료 시도
16:58:31.973 [pool-1-thread-2] taskB 완료
16:58:31.973 [pool-1-thread-1] taskA 완료
16:58:31.974 [pool-1-thread-2] taskC 시작
16:58:31.975 [pool-1-thread-1] longTask 시작
16:58:32.977 [pool-1-thread-2] taskC 완료
16:58:40.973 [      main] 서비스 정상 종료 실패 -> 강제 종료 시도
16:58:40.974 [pool-1-thread-1] 인터럽트 발생, sleep interrupted
16:58:40.975 [      main] == shutdown 완료 ==

```

```

16:58:40.975 [      main] [pool=0, active=0, queuedTasks=0, completedTasks=4]

//인터럽트에서 예외 발생
Exception in thread "pool-1-thread-1" java.lang.RuntimeException:
java.lang.InterruptedException: sleep interrupted
    at util.ThreadUtils.sleep(ThreadUtils.java:12)
    at thread.executor.RunnableTask.run(RunnableTask.java:22)
    at java.base/
java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1144
)
    at java.base/
java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:642
)
    at java.base/java.lang.Thread.run(Thread.java:1583)
Caused by: java.lang.InterruptedException: sleep interrupted
    at java.base/java.lang.Thread.sleep0(Native Method)
    at java.base/java.lang.Thread.sleep(Thread.java:509)
    at util.ThreadUtils.sleep(ThreadUtils.java:9)
    ... 4 more

```

작업 처리에 필요한 시간

- taskA, taskB, taskC: 1초
- longTask: 100초

서비스 종료

```
es.shutdown();
```

- 새로운 작업을 받지 않는다. 처리 중이거나, 큐에 이미 대기중인 작업은 처리한다. 이후에 풀의 스레드를 종료한다.
- shutdown() 은 블로킹 메서드가 아니다. 서비스가 종료될 때 까지 main 스레드가 대기하지 않는다. main 스레드는 바로 다음 코드를 호출한다.

```
if (!es.awaitTermination(10, TimeUnit.SECONDS)){...}
```

- 블로킹 메서드이다.
- main 스레드는 대기하며 서비스 종료를 10초간 기다린다.
 - 만약 10초 안에 모든 작업이 완료된다면 true를 반환한다.
- 여기서 taskA, taskB, taskC의 수행이 완료된다. 그런데 longTask는 10초가 지나도 완료되지 않았다.

- 따라서 `false`를 반환한다.

서비스 정상 종료 실패 → 강제 종료 시도

```
log("서비스 정상 종료 실패 -> 강제 종료 시도");
es.shutdownNow();
// 작업이 취소될 때 까지 대기한다.
if (!es.awaitTermination(10, TimeUnit.SECONDS)) {
    log("서비스가 종료되지 않았습니다.");
}
```

- 정상 종료가 10초 이상 너무 오래 걸렸다.
- `shutdownNow()`를 통해 강제 종료에 들어간다. `shutdown()`과 마찬가지로 블로킹 메서드가 아니다.
- 강제 종료를 하면 작업 중인 스레드에 인터럽트가 발생한다. 다음 로그를 통해 인터럽트를 확인할 수 있다.
- 인터럽트가 발생하면서 스레드도 작업을 종료하고, `shutdownNow()`를 통한 강제 shutdown도 완료된다.

```
16:58:40.973 [      main] 서비스 정상 종료 실패 -> 강제 종료 시도
16:58:40.974 [pool-1-thread-1] 인터럽트 발생, sleep interrupted
16:58:40.975 [      main] == shutdown 완료 ==
```

서비스 종료 실패

그런데 마지막에 강제 종료인 `es.shutdownNow()`를 호출한 다음에 왜 10초간 또 기다릴까?

`shutdownNow()`가 작업 중인 스레드에 인터럽트를 호출하는 것은 맞다. 인터럽트를 호출하더라도 여러가지 이유로 작업에 시간이 걸릴 수 있다. 인터럽트 이후에 자원을 정리하는 어떤 간단한 작업을 수행할 수 도 있다. 이런 시간을 기다려주는 것이다.

극단적이지만 최악의 경우 스레드가 다음과 같이 인터럽트를 받을 수 없는 코드를 수행중일 수 있다. 이 경우 인터럽트 예외가 발생하지 않고, 스레드도 계속 수행된다.

인터럽트를 받을 수 없는 코드

```
while(true) { //Empty }
```

이런 스레드는 자바를 강제 종료해야 제거할 수 있다.

이런 경우를 대비해서 강제 종료 후 10초간 대기해도 작업이 완료되지 않으면 "서비스가 종료되지 않았습니다"라고 개발자가 인지할 수 있는 로그를 남겨두어야 한다. 그래야 개발자가 나중에 문제를 찾아서 코드를 수정할 수 있다.

정리

서비스를 종료할 때 생각보다 고려해야 할 점이 많다는 점을 이해했을 것이다. 기본적으로 우아한 종료를 선택하고, 우아한 종료가 되지 않으면 무한정 기다릴 수는 없으니, 그 다음으로 강제 종료를 하는 방식으로 접근하는 것이 좋다.

Executor 스레드 풀 관리 - 코드

이번 시간에는 Executor 프레임워크가 어떤식으로 스레드를 관리하는지 깊이있게 알아보자.

이 부분을 알아두면 실무에서 대량의 요청을 별도의 스레드에서 어떤식으로 처리해야 하는지에 대한 기본기를 쌓을 수 있을 것이다.

ExecutorService의 기본 구현체인 ThreadPoolExecutor의 생성자는 다음 속성을 사용한다.

- corePoolSize: 스레드 풀에서 관리되는 기본 스레드의 수
- maximumPoolSize: 스레드 풀에서 관리되는 최대 스레드 수
- keepAliveTime, TimeUnit unit: 기본 스레드 수를 초과해서 만들어진 초과 스레드가 생존할 수 있는 대기 시간, 이 시간 동안 처리할 작업이 없다면 초과 스레드는 제거된다.
- BlockingQueue workQueue: 작업을 보관할 블로킹 큐

corePoolSize와 maximumPoolSize의 차이를 알아보기 위해 간단한 예제를 만들어보자.

먼저 예제를 좀 더 쉽게 확인하기 위해 ExecutorUtils에 메서드를 하나 추가하자.

```
package thread.executor;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.ThreadPoolExecutor;

import static util.MyLogger.log;

public class ExecutorUtils {

    public static void printState(ExecutorService executorService) {
        if (executorService instanceof ThreadPoolExecutor poolExecutor) {
            int pool = poolExecutor.getPoolSize();
            int active = poolExecutor.getActiveCount();
            int queued = poolExecutor.getQueue().size();
        }
    }
}
```

```

        long completedTask = poolExecutor.getCompletedTaskCount();
        log("[pool=" + pool + ", active=" + active + ", queuedTasks=" +
queued + ", completedTasks=" + completedTask + "]);
    } else {
        log(executorService);
    }
}

// 추가
public static void printState(ExecutorService executorService, String
taskName) {
    if (executorService instanceof ThreadPoolExecutor poolExecutor) {
        int pool = poolExecutor.getPoolSize();
        int active = poolExecutor.getActiveCount();
        int queued = poolExecutor.getQueue().size();
        long completedTask = poolExecutor.getCompletedTaskCount();
        log(taskName + " -> [pool=" + pool + ", active=" + active + ",
queuedTasks=" + queued + ", completedTasks=" + completedTask + "]);
    } else {
        log(taskName + " -> " + executorService);
    }
}
}

```

- printState() 메서드를 하나 오버로딩 했다. 단순히 taskName을 출력하는 부분이 추가되었다.
- 중복된 부분을 제거할 수 있지만, 기본 코드를 유지하기 위해 그대로 복사해서 약간만 수정했다.

추가로 이전에 만든 RunnableTask 를 사용한다. 다음 코드는 앞서 만든 코드이니 참고만 하자.

RunnableTask 는 기본 1초 정도 작업을 수행한다고 가정한다.

```

package thread.executor;

import static util.MyLogger.log;
import static util.ThreadUtils.sleep;

public class RunnableTask implements Runnable {
    private final String name;
    private int sleepMs = 1000;

    public RunnableTask(String name) {
        this.name = name;
    }
}

```

```

    }

    public RunnableTask(String name, int sleepMs) {
        this.name = name;
        this.sleepMs = sleepMs;
    }

    @Override
    public void run() {
        log(name + " 시작");
        sleep(sleepMs); // 작업 시간 시뮬레이션
        log(name + " 완료");
    }
}

```

```

package thread.executor.poolsize;

import thread.executor.RunnableTask;

import java.util.concurrent.*;

import static thread.executor.ExecutorUtils.printState;
import static util.MyLogger.log;
import static util.ThreadUtils.sleep;

public class PoolSizeMainV1 {

    public static void main(String[] args) throws InterruptedException {
        BlockingQueue<Runnable> workQueue = new ArrayBlockingQueue<>(2);
        ExecutorService es = new ThreadPoolExecutor(2, 4, 3000,
TimeUnit.MILLISECONDS, workQueue);
        printState(es);

        es.execute(new RunnableTask("task1"));
        printState(es, "task1");

        es.execute(new RunnableTask("task2"));
        printState(es, "task2");

        es.execute(new RunnableTask("task3"));
    }
}

```

```

        printState(es, "task3");

        es.execute(new RunnableTask("task4"));
        printState(es, "task4");

        es.execute(new RunnableTask("task5"));
        printState(es, "task5");

        es.execute(new RunnableTask("task6"));
        printState(es, "task6");

        try {
            es.execute(new RunnableTask("task7"));
        } catch (RejectedExecutionException e) {
            log("task7 실행 거절 예외 발생: " + e);
        }

        sleep(3000);
        log("== 작업 수행 완료 ==");
        printState(es);

        sleep(3000);
        log("== maximumPoolSize 대기 시간 초과 ==");
        printState(es);

        es.close();
        log("== shutdown 완료 ==");
        printState(es);
    }
}

```

```

BlockingQueue<Runnable> workQueue = new ArrayBlockingQueue<>(2);
ExecutorService es = new ThreadPoolExecutor(2, 4, 3000, TimeUnit.MILLISECONDS,
workQueue);

```

- 작업을 보관할 블로킹 큐의 구현체로 `ArrayBlockingQueue(2)` 를 사용했다. 사이즈를 2로 설정했으므로 최대 2개까지 작업을 큐에 보관할 수 있다.
- `corePoolSize=2`, `maximumPoolSize=4` 를 사용해서 기본 스레드는 2개, 최대 스레드는 4개로 설정했다.
 - 스레드 풀에 기본 2개의 스레드를 운영한다. 요청이 너무 많거나 급한 경우 스레드 풀은 최대 4개까지 스레

드를 증가시켜서 사용할 수 있다. 이렇게 기본 스레드 수를 초과해서 만들어진 스레드를 **초과 스레드**라 하겠다.

- `3000, TimeUnit.MILLISECONDS`
 - 초과 스레드가 생존할 수 있는 대기 시간을 뜻한다. 이 시간 동안 초과 스레드가 처리할 작업이 없다면 초과 스레드는 제거된다.
 - 여기서는 3000 밀리초(3초)를 설정했으므로, 초과 스레드가 3초간 작업을 하지 않고 대기한다면 초과 스레드는 스레드 풀에서 제거된다.

실행 결과

```
11:36:23.260 [      main] [pool=0, active=0, queuedTasks=0, completedTasks=0]
11:36:23.263 [pool-1-thread-1] task1 시작
11:36:23.267 [      main] task1 -> [pool=1, active=1, queuedTasks=0,
completedTasks=0]
11:36:23.267 [      main] task2 -> [pool=2, active=2, queuedTasks=0,
completedTasks=0]
11:36:23.267 [pool-1-thread-2] task2 시작
11:36:23.267 [      main] task3 -> [pool=2, active=2, queuedTasks=1,
completedTasks=0]
11:36:23.268 [      main] task4 -> [pool=2, active=2, queuedTasks=2,
completedTasks=0]
11:36:23.268 [      main] task5 -> [pool=3, active=3, queuedTasks=2,
completedTasks=0]
11:36:23.268 [pool-1-thread-3] task5 시작
11:36:23.268 [      main] task6 -> [pool=4, active=4, queuedTasks=2,
completedTasks=0]
11:36:23.268 [pool-1-thread-4] task6 시작
11:36:23.268 [      main] task7 실행 거절 예외 발생:
java.util.concurrent.RejectedExecutionException: Task
thread.executor RunnableTask@3abbfa04 rejected from
java.util.concurrent.ThreadPoolExecutor@7f690630[Running, pool size = 4,
active threads = 4, queued tasks = 2, completed tasks = 0]
11:36:24.268 [pool-1-thread-1] task1 완료
11:36:24.268 [pool-1-thread-1] task3 시작
11:36:24.269 [pool-1-thread-3] task5 완료
11:36:24.269 [pool-1-thread-3] task4 시작
11:36:24.269 [pool-1-thread-2] task2 완료
11:36:24.269 [pool-1-thread-4] task6 완료
11:36:25.273 [pool-1-thread-1] task3 완료
11:36:25.273 [pool-1-thread-3] task4 완료
11:36:26.273 [      main] == 작업 수행 완료 ==
11:36:26.273 [      main] [pool=4, active=0, queuedTasks=0, completedTasks=6]
```

```
11:36:29.276 [      main] == maximumPoolSize 대기 시간 초과 ==  
11:36:29.277 [      main] [pool=2, active=0, queuedTasks=0, completedTasks=6]  
11:36:29.278 [      main] == shutdown 완료 ==  
11:36:29.278 [      main] [pool=0, active=0, queuedTasks=0, completedTasks=6]
```

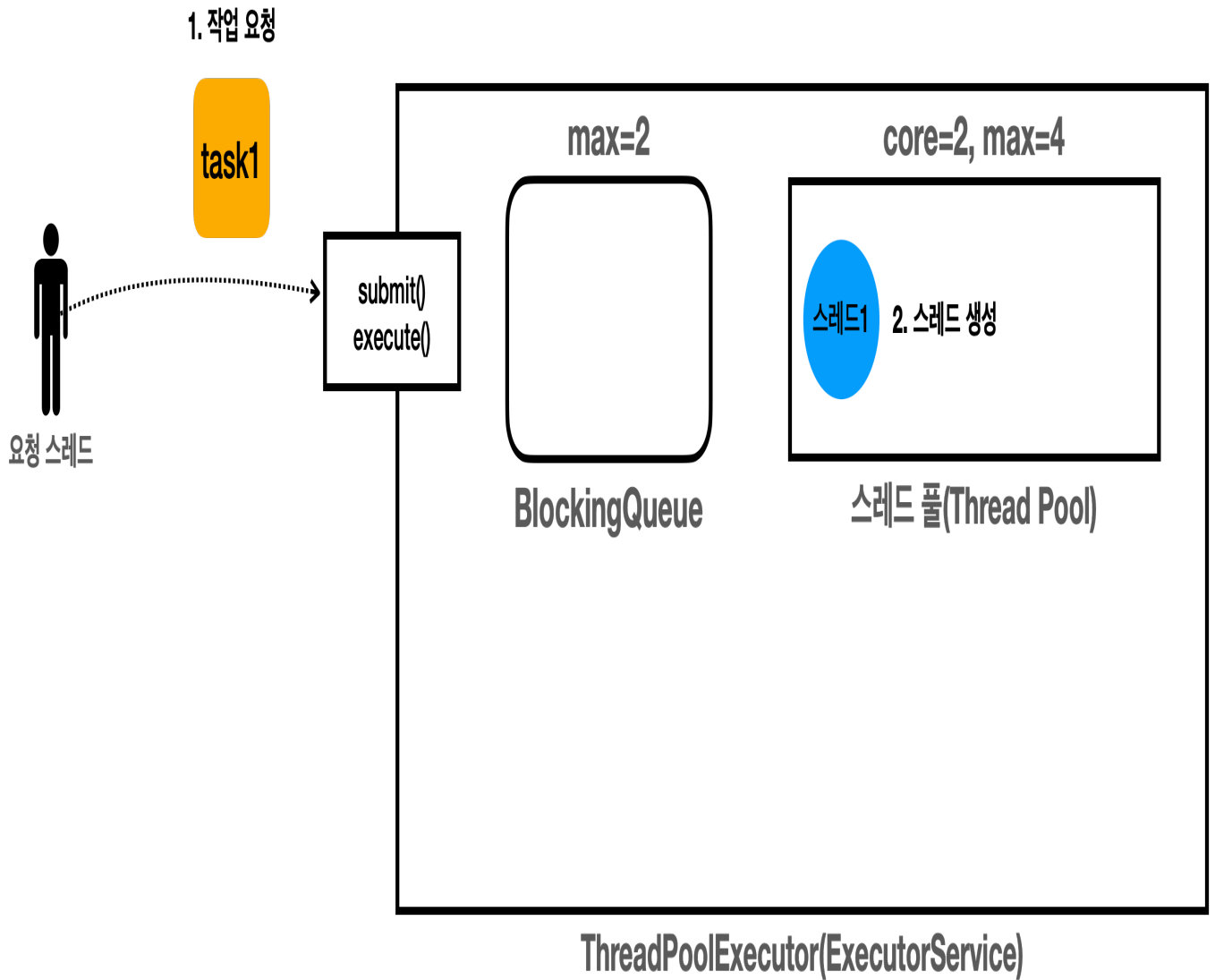
작업을 전달한 다음에 로그를 출력하기 때문에 로그 출력의 순서가 더 늦을 수 있다.

```
es.execute(new RunnableTask("task1")); // 작업 전달  
printStats(es, "task1"); // 로그 출력
```

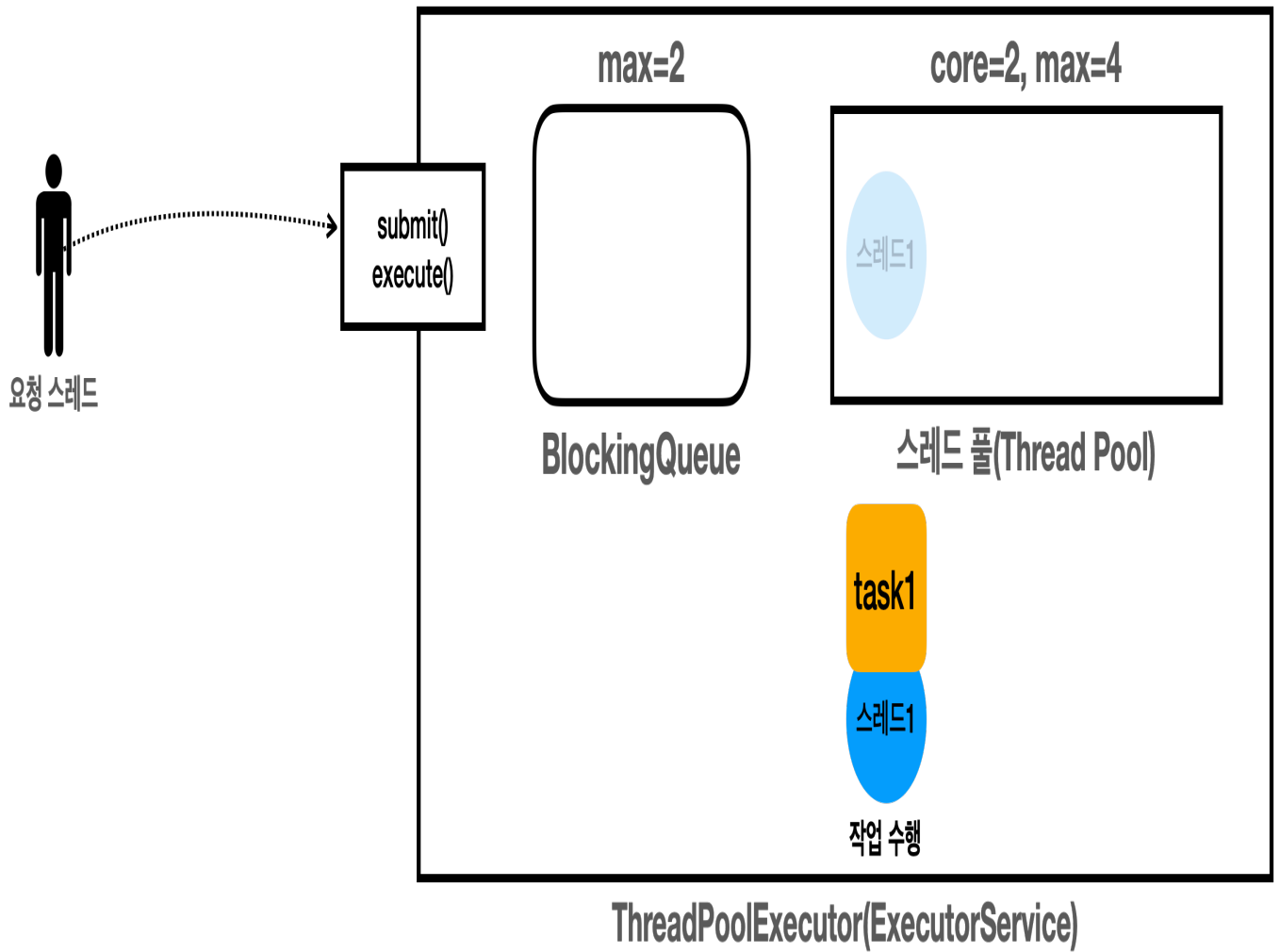
```
11:36:23.263 [pool-1-thread-1] task1 시작  
11:36:23.267 [      main] task1 -> [pool=1, active=1, queuedTasks=0,  
completedTasks=0]
```

Executor 스레드 풀 관리 - 분석

실행 분석

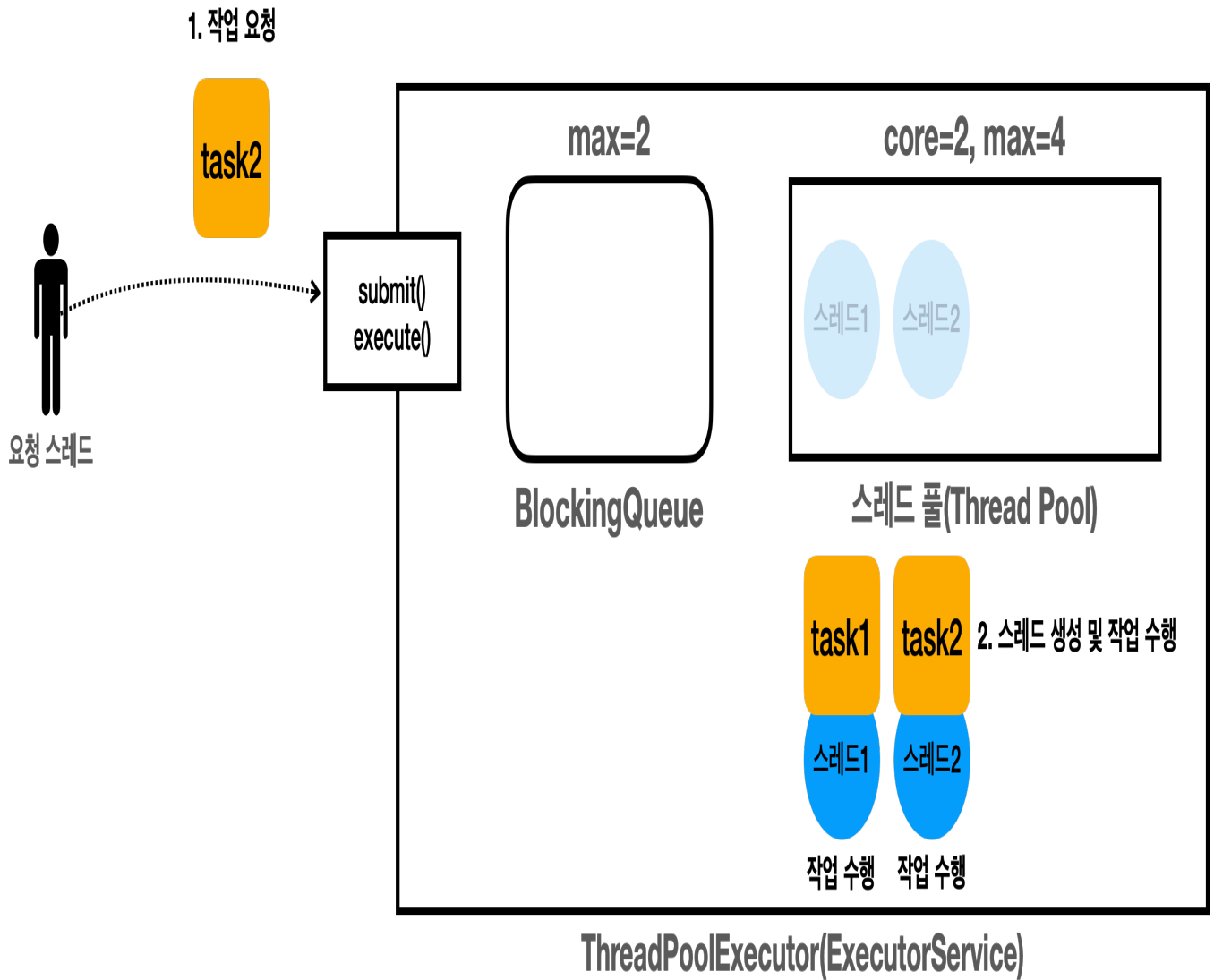


- task1 작업을 요청한다.
- Executor 는 스레드 풀에 스레드가 core 사이즈 만큼 있는지 확인한다.
 - core 사이즈 만큼 없다면 스레드를 하나 생성한다.
 - 작업을 처리하기 위해 스레드를 하나 생성했기 때문에 작업을 큐에 넣을 필요 없이, 해당 스레드가 바로 작업을 처리한다.



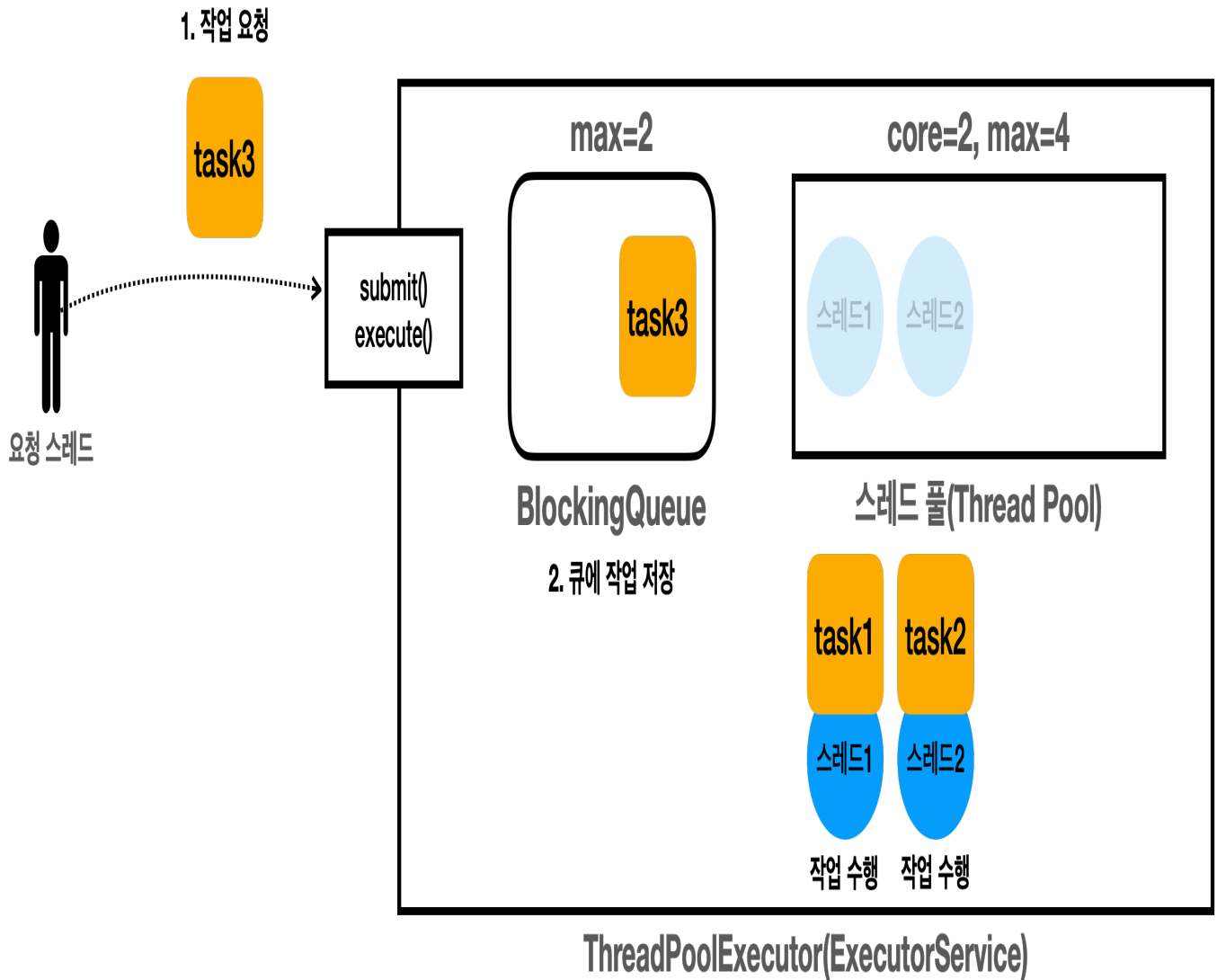
```
11:10:14.503 [pool-1-thread-1] task1 시작
11:10:14.508 [      main] task1 -> [pool=1, active=1, queuedTasks=0,
completedTasks=0]
```

- 새로 만들어진 스레드1이 task1 을 수행한다.



```
11:36:23.267 [    main] task2 -> [pool=2, active=2, queuedTasks=0,
completedTasks=0]
11:36:23.267 [pool-1-thread-2] task2 시작
```

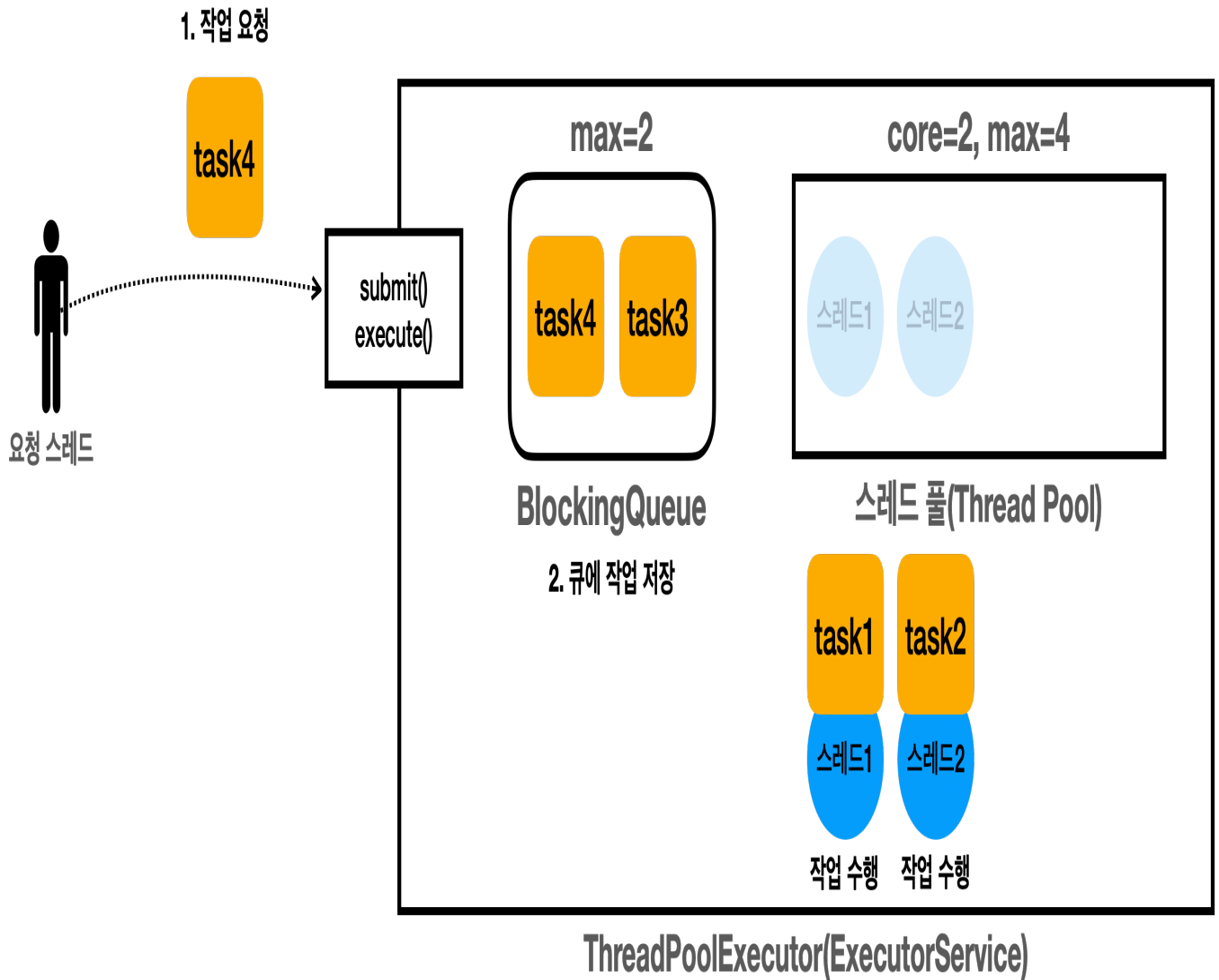
- task2 를 요청한다.
- Executor 는 스레드 풀에 스레드가 core 사이즈 만큼 있는지 확인한다.
 - 아직 core 사이즈 만큼 없으므로 스레드를 하나 생성한다.
- 새로 만들어진 스레드2가 task2 를 처리한다.



스레드 풀의 스레드 수가 core 사이즈를 넘어가면 작업을 큐에 보관

```
11:36:23.267 [    main] task3 -> [pool=2, active=2, queuedTasks=1,
completedTasks=0]
```

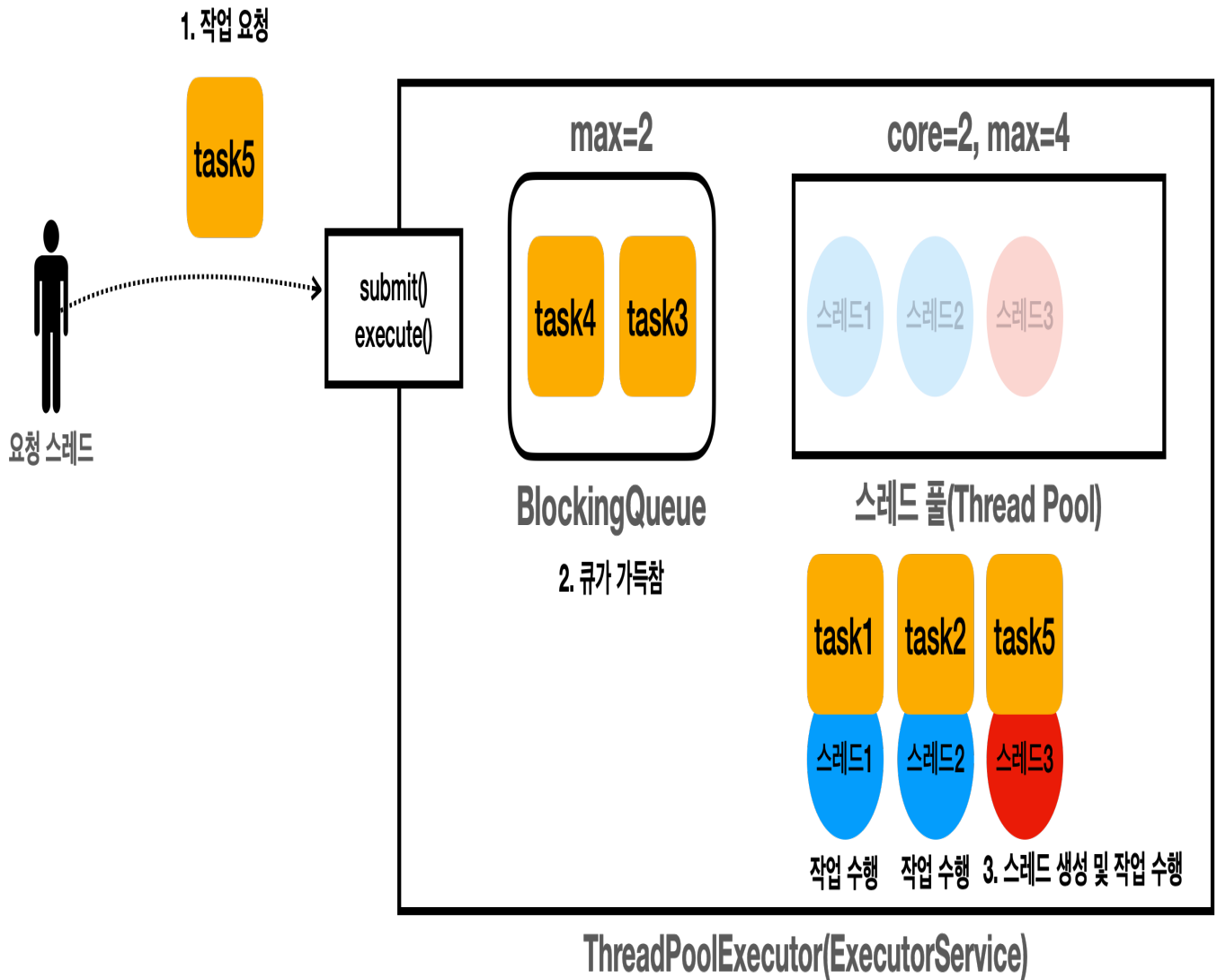
- task3 작업을 요청한다.
- Executor 는 스레드 풀에 스레드가 core 사이즈 만큼 있는지 확인한다.
- core 사이즈 만큼 스레드가 이미 만들어져 있고, 스레드 풀에 사용할 수 있는 스레드가 없으므로 이 경우 큐에 작업을 보관한다.



스레드 풀의 스레드 수가 core 사이즈를 넘어가면 작업을 큐에 보관

```
11:36:23.268 [    main] task4 -> [pool=2, active=2, queuedTasks=2,
completedTasks=0]
```

- task4 작업을 요청한다.
- Executor 는 스레드 풀에 스레드가 core 사이즈 만큼 있는지 확인한다.
- core 사이즈 만큼 스레드가 이미 만들어져 있고, 스레드 풀에 사용할 수 있는 스레드가 없으므로 이 경우 큐에 작업을 보관한다.



큐가 가득차면 max 사이즈까지는 추가 스레드를 생성해서 수행

```
11:36:23.268 [    main] task5 -> [pool=3, active=3, queuedTasks=2,
completedTasks=0]
11:36:23.268 [pool-1-thread-3] task5 시작
```

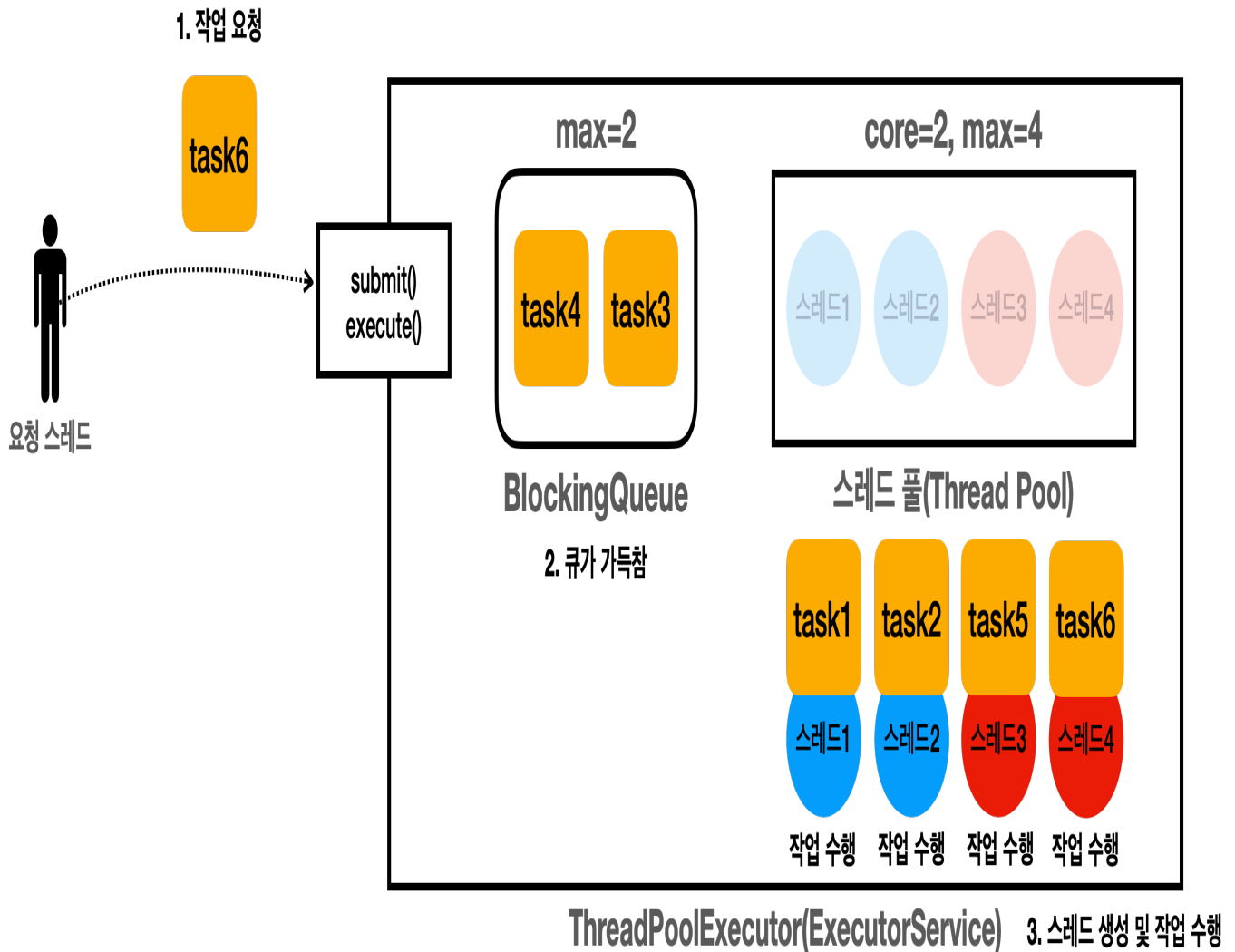
- task5 작업을 요청한다.
- Executor 는 스레드 풀에 스레드가 core 사이즈 만큼 있는지 확인한다. → core 사이즈 만큼 있다.
- Executor 는 큐에 보관을 시도한다 → 큐가 가득 찼다.

큐가 가득차면 긴급 상황이다. 대기하는 작업이 꽤 찰 정도로 요청이 많다는 뜻이다.

이 경우 Executor 는 max(maximumPoolSize) 사이즈까지 초과 스레드를 만들어서 작업을 수행한다.

- core=2 : 기본 스레드는 최대 2개
- max=4 : 기본 스레드 2개에 초과 스레드 2개 합계 총 4개 가능 (초과 스레드 = max - core)
- Executor 는 초과 스레드인 스레드3을 만든다.

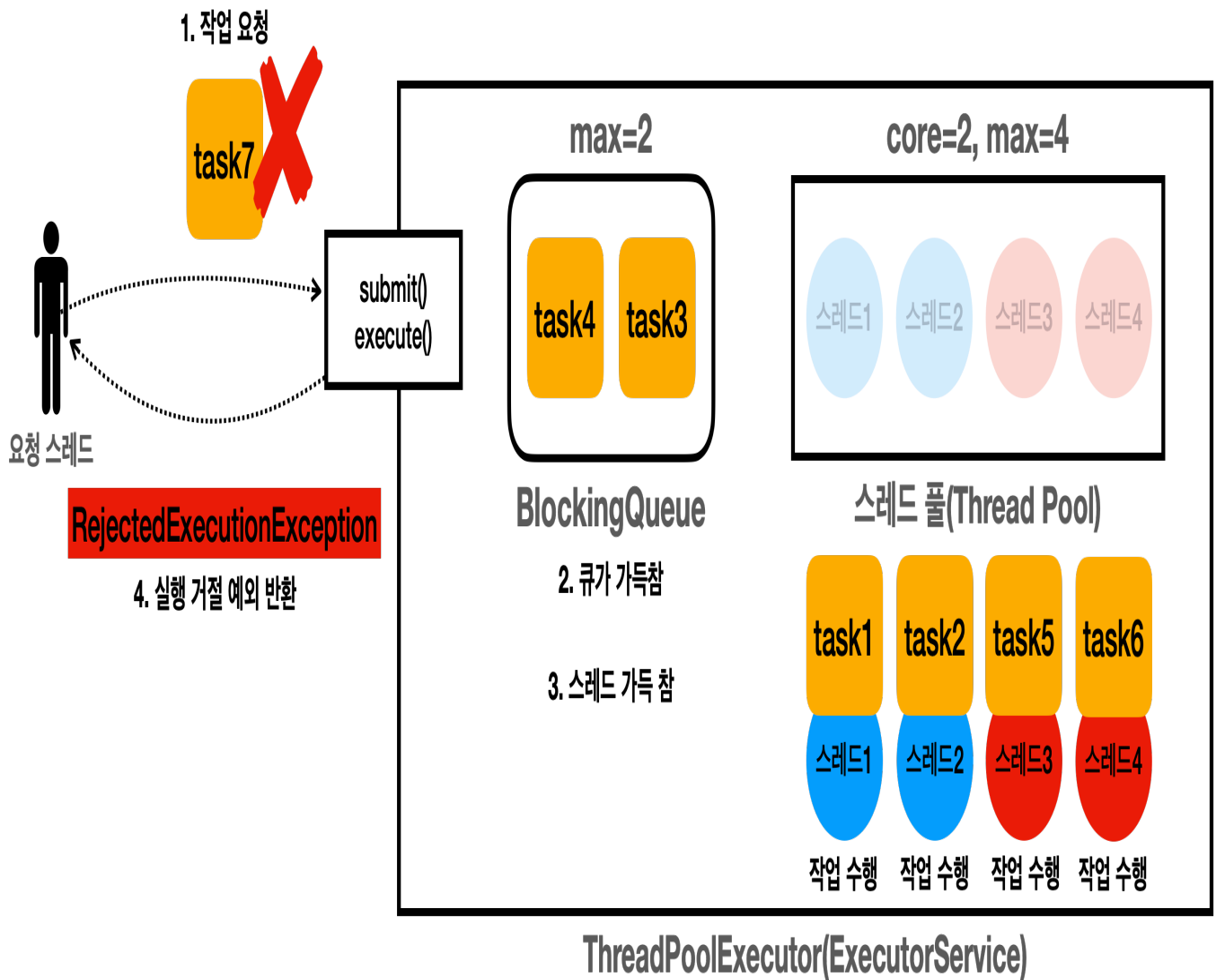
- 작업을 처리하기 위해 스레드를 하나 생성했기 때문에 작업을 큐에 넣을 필요 없이, 해당 스레드가 바로 작업을 처리한다.
 - 참고로 이 경우 큐가 가득차기 때문에 큐에 넣는 것도 불가능하다.
- 스레드3이 task5 를 처리한다.



큐가 가득차면 max 사이즈까지는 추가 스레드를 생성해서 수행

```
11:36:23.268 [    main] task6 -> [pool=4, active=4, queuedTasks=2,
completedTasks=0]
11:36:23.268 [pool-1-thread-4] task6 시작
```

- task6 작업을 요청한다.
- 큐가 가득차다.
- Executor 는 초과 스레드인 스레드4를 만들어서 task6 을 처리한다.
 - 큐가 가득차기 때문에 작업을 큐에 넣는 것은 불가능하다.

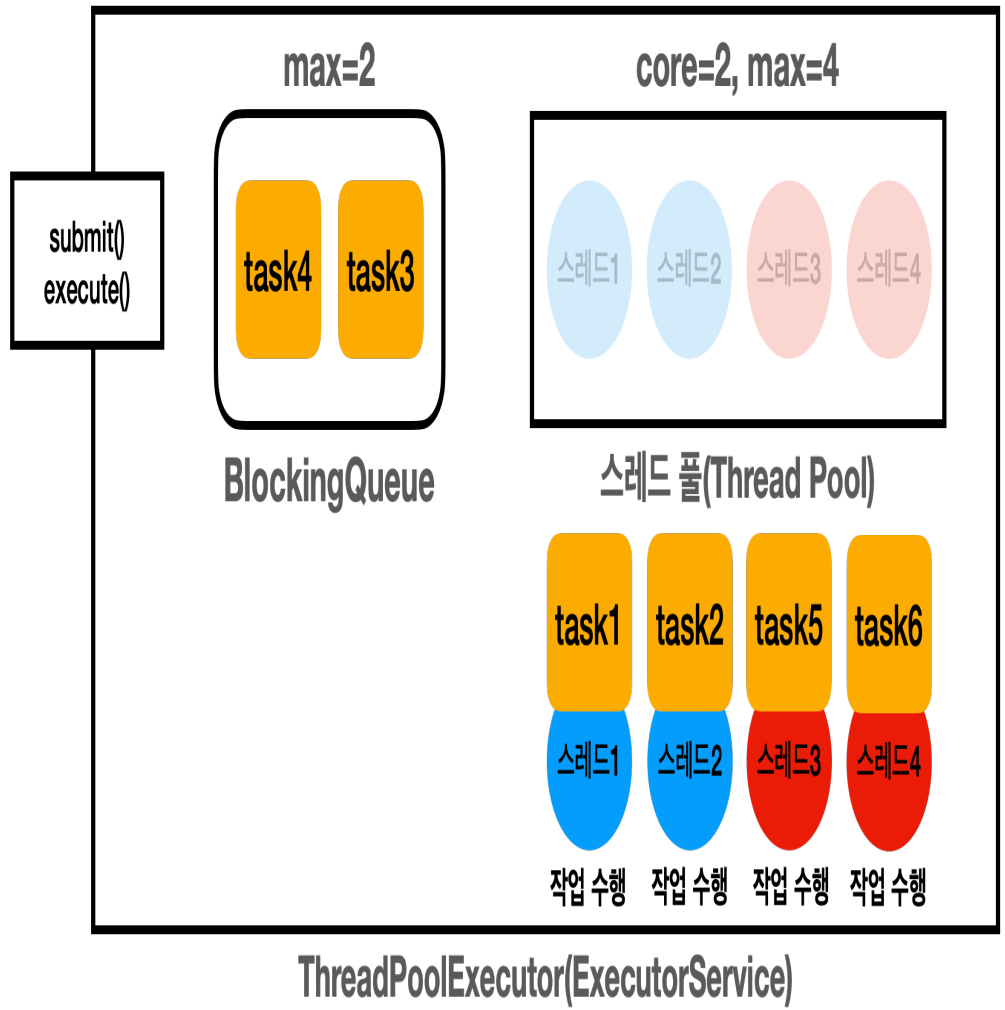
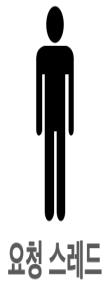


큐도 가득차고 스레드 풀의 스레드도 max까지 가득 차면 -> 실행 거절 예외

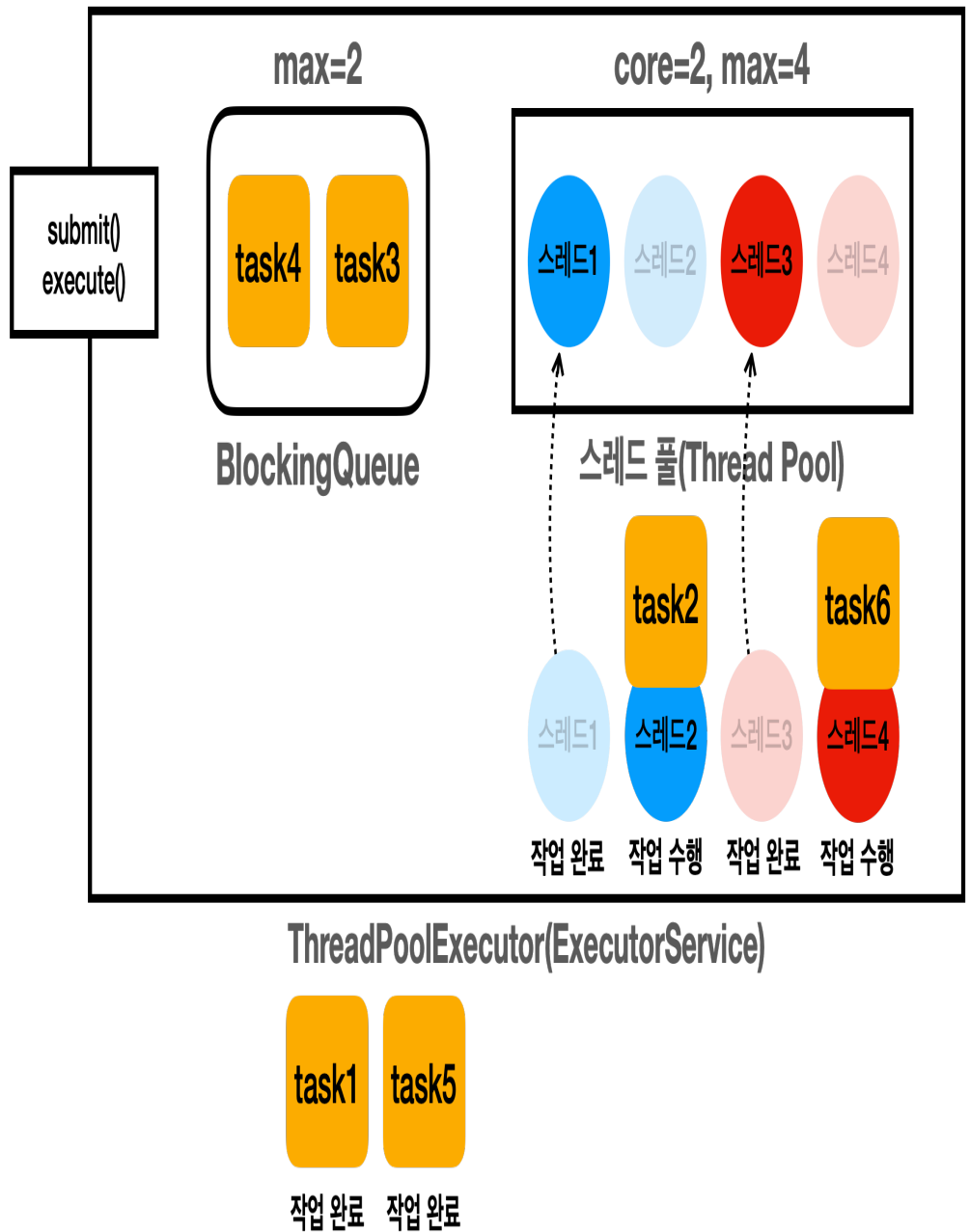
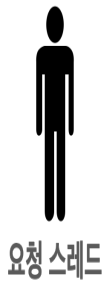
```
11:36:23.268 [      main] task7 실행 거절 예외 발생:
java.util.concurrent.RejectedExecutionException: Task
thread.executor RunnableTask@3abbfa04 rejected from
java.util.concurrent.ThreadPoolExecutor@7f690630[Running, pool size = 4,
active threads = 4, queued tasks = 2, completed tasks = 0]
```

- task7 작업을 요청한다.
- 큐가 가득차다.
- 스레드 풀의 스레드도 max 사이즈 만큼 가득차다.
- RejectedExecutionException 이 발생한다.

이 경우 큐에 넣을 수도 없고, 작업을 수행할 스레드도 만들 수 없다. 따라서 작업을 거절한다.

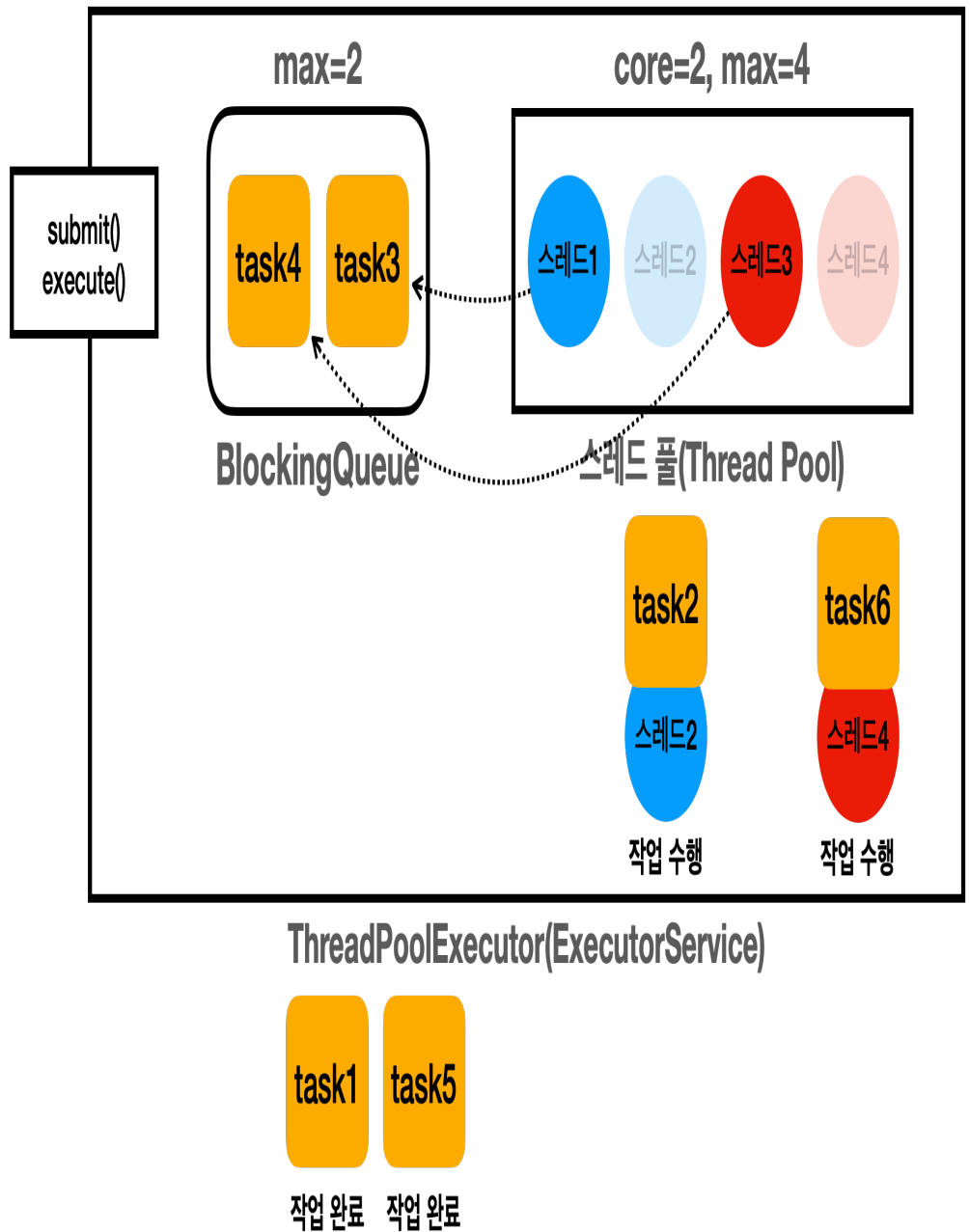
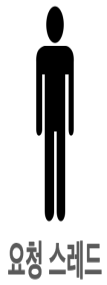


- 작업들이 수행된다.

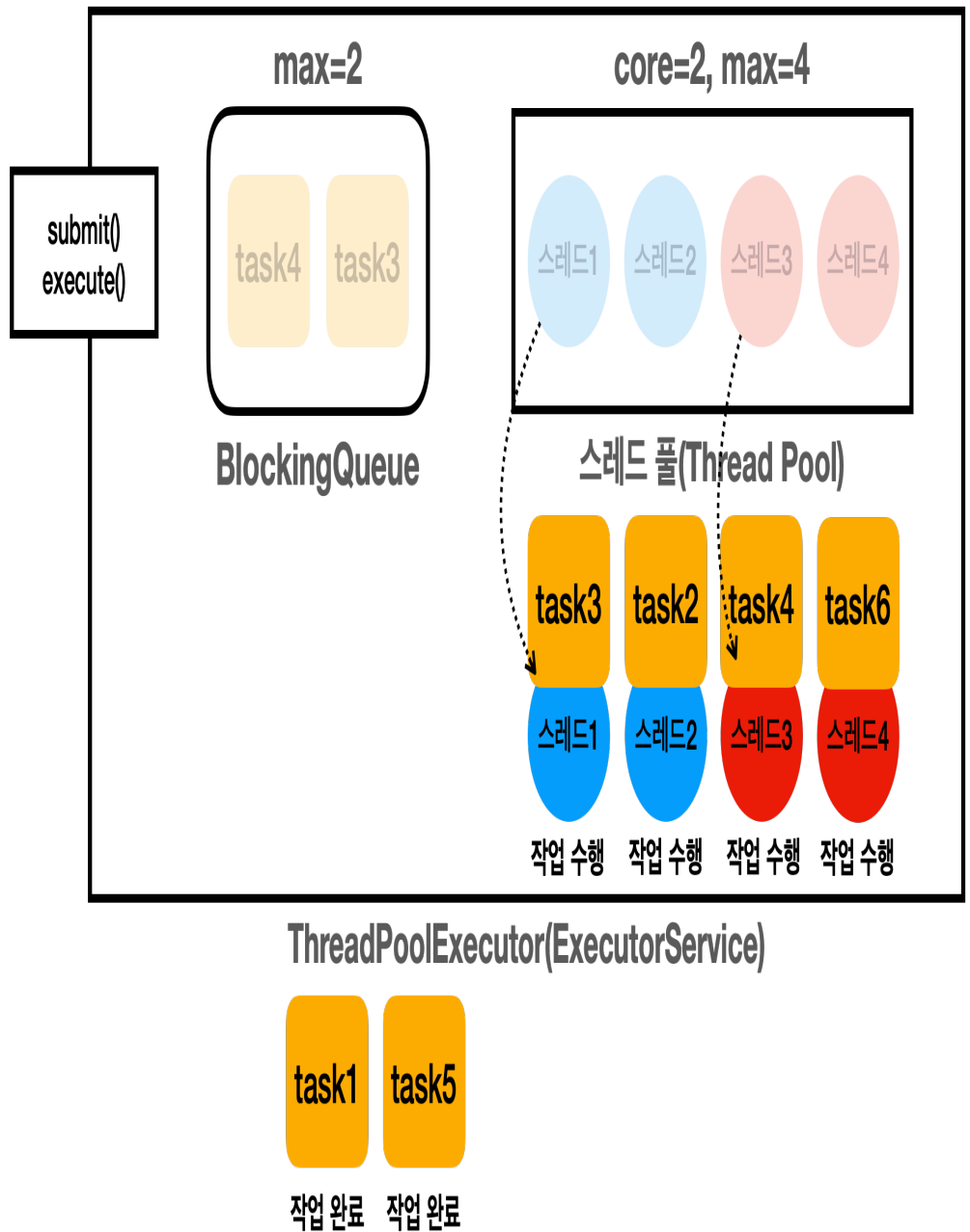
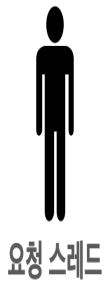


```
11:36:24.268 [pool-1-thread-1] task1 완료
11:36:24.269 [pool-1-thread-3] task5 완료
```

- 스레드 1이 task1 을 스레드 3이 task5 의 작업을 완료하고 스레드 풀에 대기 상태로 돌아간다.

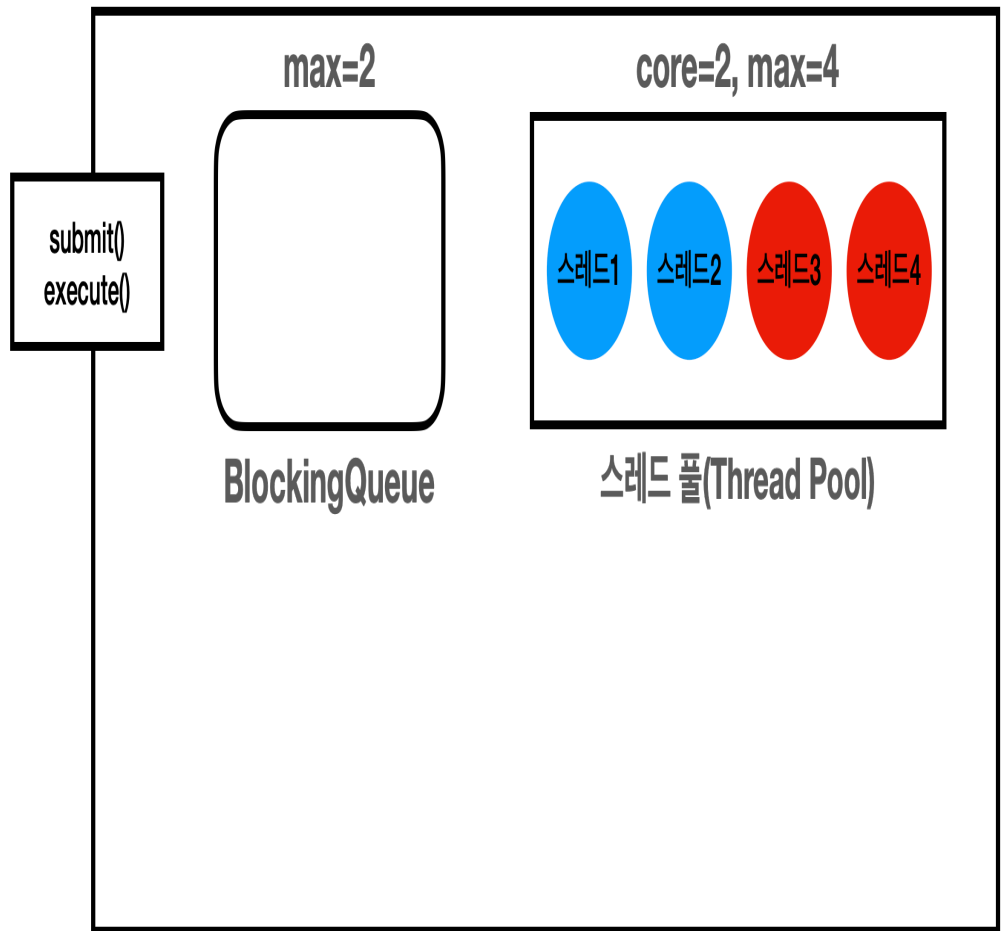
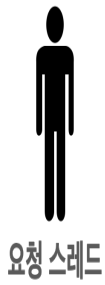


- 스레드 풀의 스레드는 큐의 데이터를 획득하기 위해 대기한다.
- 스레드1: task3 을 획득한다.
- 스레드3: task4 를 획득한다.



```
11:36:24.268 [pool-1-thread-1] task3 시작
11:36:24.269 [pool-1-thread-3] task4 시작
```

- 계속 작업을 수행한다.

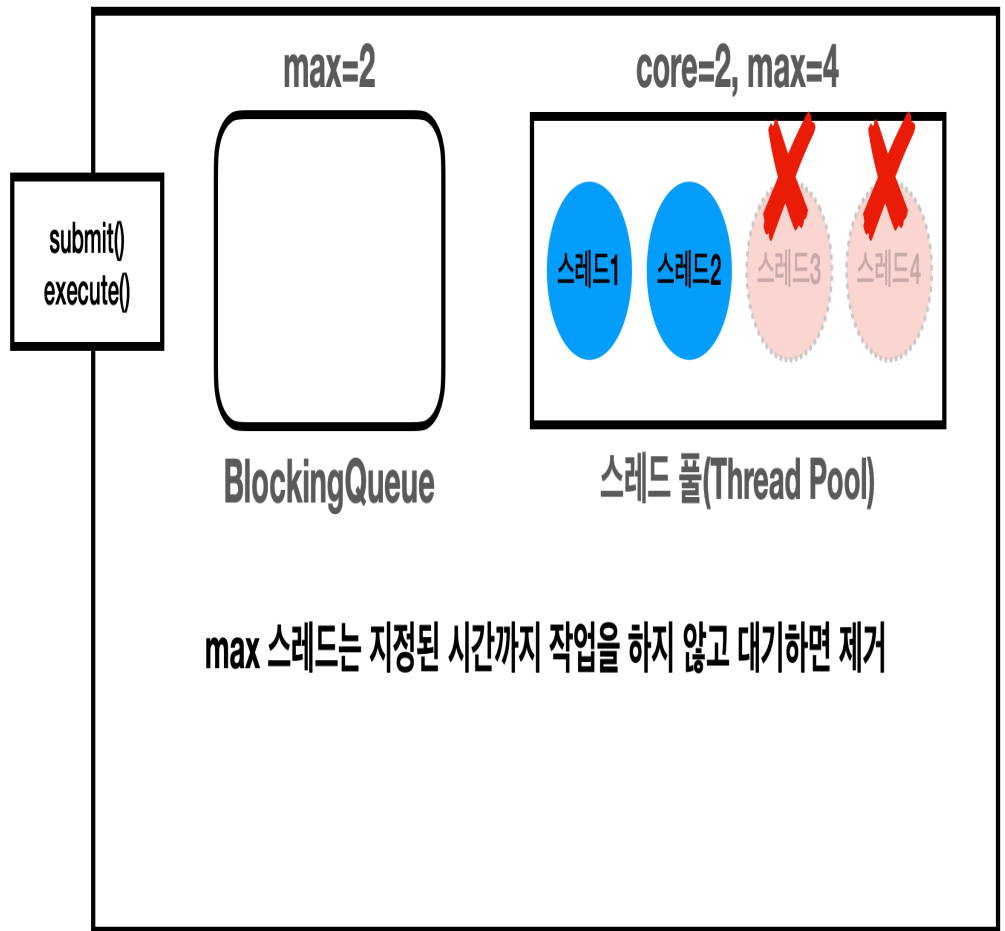
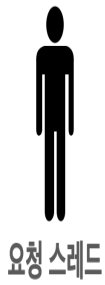


ThreadPoolExecutor(ExecutorService)



```
11:36:24.269 [pool-1-thread-2] task2 완료
11:36:24.269 [pool-1-thread-4] task6 완료
11:36:25.273 [pool-1-thread-1] task3 완료
11:36:25.273 [pool-1-thread-3] task4 완료
11:36:26.273 [      main] == 작업 수행 완료 ==
11:36:26.273 [      main] [pool=4, active=0, queuedTasks=0, completedTasks=6]
```

- 모든 작업이 완료된다.

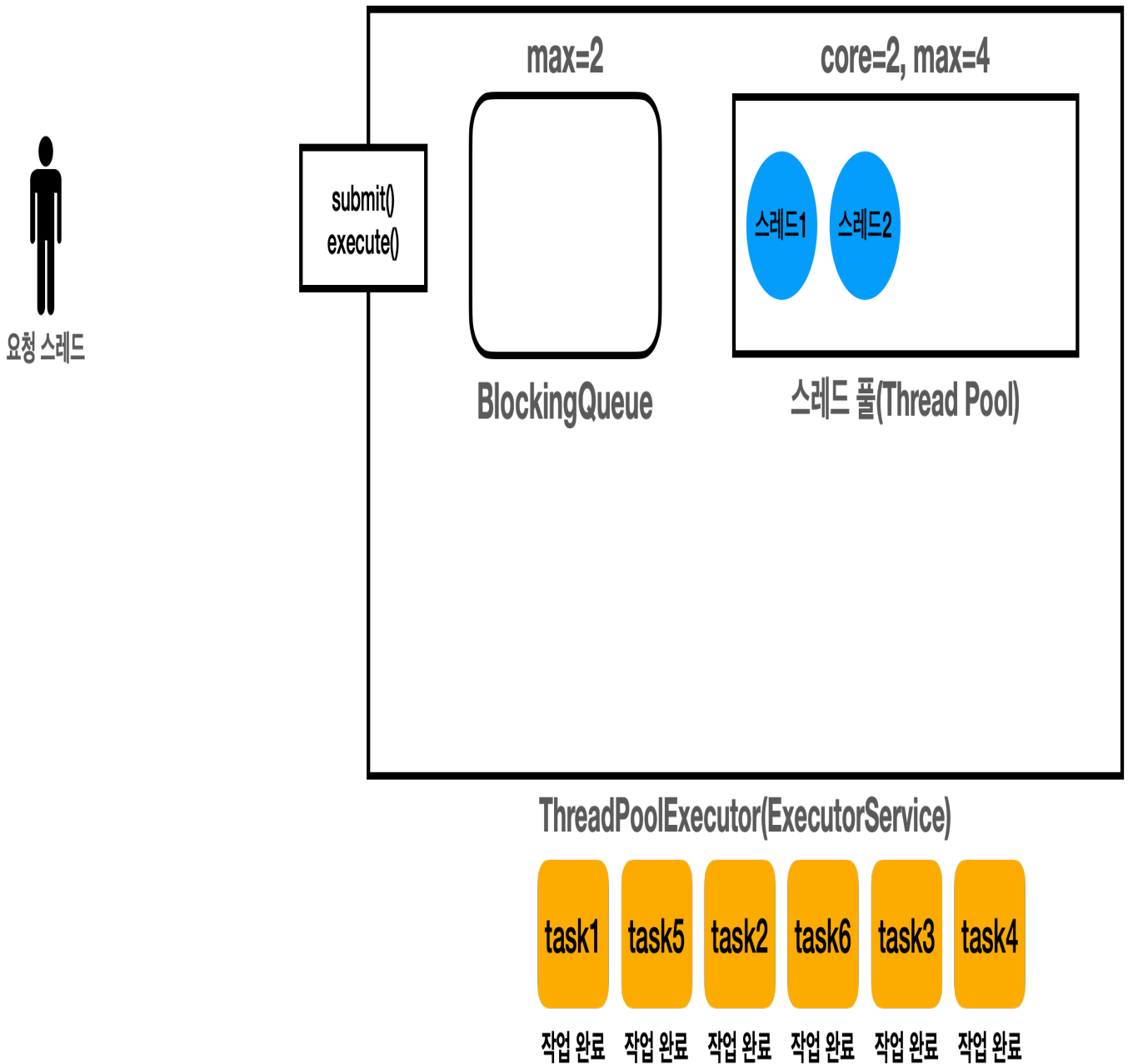


ThreadPoolExecutor(ExecutorService)



```
11:36:29.276 [    main] == maximumPoolSize 대기 시간 초과 ==  
11:36:29.277 [    main] [pool=2, active=0, queuedTasks=0, completedTasks=6]
```

- 스레드3, 스레드4와 같은 초과 스레드들은 지정된 시간까지 작업을 하지 않고 대기하면 제거된다. 긴급한 작업들이 끝난 것으로 이해하면 된다.
- 여기서는 지정한 3초간 스레드3, 스레드4가 작업을 진행하지 않았기 때문에 스레드 풀에서 제거된다.
- 참고로 초과 스레드가 작업을 처리할 때 마다 시간은 계속 초기화 된다.
 - 작업 요청이 계속 들어온다면 긴급한 상황이 끝난 것이 아니다. 따라서 긴급한 상황이 끝날 때 까지는 초과 스레드를 살려두는 것이 많은 스레드를 사용해서 작업을 더 빨리 처리할 수 있다.



- 초과 스레드가 제거된 모습이다.
- 이후에 `shutdown()` 이 진행되면 풀의 스레드도 모두 제거된다.

정리 - Executor 스레드 풀 관리

1. 작업을 요청하면 core 사이즈 만큼 스레드를 만든다.
2. core 사이즈를 초과하면 큐에 작업을 넣는다.
3. 큐를 초과하면 max 사이즈 만큼 스레드를 만든다. 임시로 사용되는 초과 스레드가 생성된다.
 - 큐가 가득차서 큐에 넣을 수도 없다. 초과 스레드가 바로 수행해야 한다.
4. max 사이즈를 초과하면 요청을 거절한다. 예외가 발생한다.
 - 큐도 가득차고, 풀에 최대 생성 가능한 스레드 수도 가득 찼다. 작업을 받을 수 없다.

스레드 미리 생성하기

응답시간이 아주 중요한 서버라면, 서버가 고객의 처음 요청을 받기 전에 스레드를 스레드 풀에 미리 생성해두고 싶을 수 있다.

스레드를 미리 생성해두면, 처음 요청에서 사용되는 스레드의 생성 시간을 줄일 수 있다.

`ThreadPoolExecutor.prestartAllCoreThreads()` 를 사용하면 기본 스레드를 미리 생성할 수 있다.

참고로 `ExecutorService` 는 이 메서드를 제공하지 않는다.

```
package thread.executor;

import java.util.concurrent.*;

import static thread.executor.ExecutorUtils.*;
import static util.ThreadUtils.sleep;

public class PrestartPoolMain {

    public static void main(String[] args) {
        ExecutorService es = Executors.newFixedThreadPool(1000);

        printState(es);
        ThreadPoolExecutor poolExecutor = (ThreadPoolExecutor) es;
        poolExecutor.prestartAllCoreThreads();
        printState(es);
    }
}
```

```
12:33:34.971 [      main] [pool=0, active=0, queuedTasks=0, completedTask=0]
12:33:35.117 [      main] [pool=1000, active=0, queuedTasks=0, completedTask=0]
```

Executor 전략 - 고정 풀 전략

Executor 스레드 풀 관리 - 다양한 전략

`ThreadPoolExecutor` 를 사용하면 스레드 풀에 사용되는 숫자와 블로킹 큐 등 다양한 속성을 조절할 수 있다.

- `corePoolSize`: 스레드 풀에서 관리되는 기본 스레드의 수
- `maximumPoolSize`: 스레드 풀에서 관리되는 최대 스레드 수
- `keepAliveTime, TimeUnit unit`: 기본 스레드 수를 초과해서 만들어진 스레드가 생존할 수 있는 대기 시간, 이 시간 동안 처리할 작업이 없다면 초과 스레드는 제거된다.
- `BlockingQueue workQueue`: 작업을 보관할 블로킹 큐

이런 속성들을 조절하면 자신에게 맞는 스레드 풀 전략을 사용할 수 있다.

자바는 `Executors` 클래스를 통해 3가지 기본 전략을 제공한다.

- `newSingleThreadPool()`: 단일 스레드 풀 전략
- `newFixedThreadPool(nThreads)`: 고정 스레드 풀 전략
- `newCachedThreadPool()`: 캐시 스레드 풀 전략

`newSingleThreadPool()`: 단일 스레드 풀 전략

- 스레드 풀에 기본 스레드 1개만 사용한다.
- 큐 사이즈에 제한이 없다. (`LinkedBlockingQueue`)
- 주로 간단히 사용하거나, 테스트 용도로 사용한다.

```
new ThreadPoolExecutor(1, 1, 0L, TimeUnit.MILLISECONDS,
                        new LinkedBlockingQueue<Runnable>())
```

고정 스레드 풀 전략과, 캐시 스레드 풀 전략을 자세히 알아보자.

Executor 스레드 풀 관리 - 고정 풀 전략

`newFixedThreadPool(nThreads)`

- 스레드 풀에 `nThreads` 만큼의 기본 스레드를 생성한다. 초과 스레드는 생성하지 않는다.
- 큐 사이즈에 제한이 없다. (`LinkedBlockingQueue`)
- 스레드 수가 고정되어 있기 때문에 CPU, 메모리 리소스가 어느정도 예측 가능한 안정적인 방식이다.

```
new ThreadPoolExecutor(nThreads, nThreads, 0L, TimeUnit.MILLISECONDS,
                        new LinkedBlockingQueue<Runnable>())
```

코드로 실행해보자.

```
package thread.executor.poolsize;

import thread.executor RunnableTask;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

import static thread.executor.ExecutorUtils.printState;
import static util.MyLogger.log;

public class PoolSizeMainV2 {

    public static void main(String[] args) throws InterruptedException {

        ExecutorService es = Executors.newFixedThreadPool(2);
        //ExecutorService es = new ThreadPoolExecutor(2, 2, 0L,
        TimeUnit.MILLISECONDS, new LinkedBlockingQueue<>());
        log("pool 생성");
        printState(es);

        for (int i = 1; i <= 6; i++) {
            String taskName = "task" + i;
            es.execute(new RunnableTask(taskName));
            printState(es, taskName);
        }
        es.close();
        log("== shutdown 완료 ==");
    }

}
```

실행 결과

```
15:41:31.836 [      main] pool 생성
15:41:31.847 [      main] [pool=0, active=0, queuedTasks=0, completedTasks=0]
15:41:31.849 [pool-1-thread-1] task1 시작
15:41:31.854 [      main] task1 -> [pool=1, active=1, queuedTasks=0,
completedTasks=0]
```



```
15:41:31.854 [      main] task2 -> [pool=2, active=2, queuedTasks=0,
completedTasks=0]
15:41:31.854 [pool-1-thread-2] task2 시작
15:41:31.854 [      main] task3 -> [pool=2, active=2, queuedTasks=1,
completedTasks=0]
15:41:31.854 [      main] task4 -> [pool=2, active=2, queuedTasks=2,
completedTasks=0]
15:41:31.854 [      main] task5 -> [pool=2, active=2, queuedTasks=3,
completedTasks=0]
15:41:31.854 [      main] task6 -> [pool=2, active=2, queuedTasks=4,
completedTasks=0]
15:41:32.855 [pool-1-thread-1] task1 완료
15:41:32.856 [pool-1-thread-1] task3 시작
15:41:32.859 [pool-1-thread-2] task2 완료
15:41:32.859 [pool-1-thread-2] task4 시작
15:41:33.860 [pool-1-thread-1] task3 완료
15:41:33.861 [pool-1-thread-1] task5 시작
15:41:33.863 [pool-1-thread-2] task4 완료
15:41:33.863 [pool-1-thread-2] task6 시작
15:41:34.863 [pool-1-thread-2] task6 완료
15:41:34.863 [pool-1-thread-1] task5 완료
15:41:34.864 [      main] == shutdown 완료 ==
```

2개의 스레드가 안정적으로 작업을 처리하는 것을 확인할 수 있다.

이 전략은 다음과 같은 특징이 있다.

특징

스레드 수가 고정되어 있기 때문에 CPU, 메모리 리소스가 어느정도 예측 가능한 안정적인 방식이다. 큐 사이즈도 제한이 없어서 작업을 많이 담아두어도 문제가 없다.

주의

이 방식의 가장 큰 장점은 스레드 수가 고정되어서 CPU, 메모리 리소스가 어느정도 예측 가능하다는 점이다. 따라서 일반적인 상황에 가장 안정적으로 서비스를 운영할 수 있다. 하지만 상황에 따라 장점이 가장 큰 단점이 되기도 한다.

상황1 - 점진적인 사용자 확대

- 개발한 서비스가 잘 되어서 사용자가 점점 늘어난다.
- 고정 스레드 전략을 사용해서 서비스를 안정적으로 잘 운영했는데, 언젠가부터 사용자들이 서비스 응답이 점점 느리진다고 항의한다.

상황2 - 갑작스런 요청 증가

- 마케팅 팀의 이벤트가 대성공 하면서 갑자기 사용자가 폭증했다.
- 고객은 응답을 받지 못한다고 항의한다.

확인

- 개발자는 급하게 CPU, 메모리 사용량을 확인해보는데, 아무런 문제 없이 여유있고, 안정적으로 서비스가 운영되고 있다.
- 고정 스레드 전략은 실행되는 스레드 수가 고정되어 있다. 따라서 사용자가 늘어나도 CPU, 메모리 사용량이 늘어나지 않는다.
- 큐의 사이즈를 확인해보니 요청이 수 만 건이 쌓여있다. 요청이 처리되는 시간보다 쌓이는 시간이 더 빠른 것이다. 참고로 고정 풀 전략의 큐 사이즈는 무한이다.
- 예를 들어서 큐에 10000건이 쌓여있는데, 고정 스레드 수가 10이고, 각 스레드가 작업을 하나 처리하는데 1초가 걸린다면 모든 작업을 다 처리하는데는 1000초가 걸린다. 만약 처리 속도보다 작업이 쌓이는 속도가 더 빠른 경우에는 더 문제가 된다.
- 서비스 초기에는 사용자가 적기 때문에 이런 문제가 없지만, 사용자가 늘어나면 문제가 될 수 있다.
- 갑작스런 요청 증가도 물론 마찬가지이다.

결국 서버 자원은 여유가 있는데, 사용자만 점점 느려지는 문제가 발생한 것이다.

Executor 전략 - 캐시 풀 전략

newCachedThreadPool()

- 기본 스레드를 사용하지 않고, 60초 생존 주기를 가진 초과 스레드만 사용한다.
- 초과 스레드의 수는 제한이 없다.
- 큐에 작업을 저장하지 않는다. (SynchronousQueue)
 - 대신에 생산자의 요청을 스레드 풀의 소비자 스레드가 직접 받아서 바로 처리한다.
- 모든 요청이 대기하지 않고 스레드가 바로바로 처리한다. 따라서 빠른 처리가 가능하다.

```
new ThreadPoolExecutor(0, Integer.MAX_VALUE, 60L, TimeUnit.SECONDS,  
    new SynchronousQueue<Runnable>());
```

SynchronousQueue 는 아주 특별한 블로킹 큐이다.

- `BlockingQueue` 인터페이스의 구현체 중 하나이다.
- 이 큐는 내부에 저장 공간이 없다. 대신에 생산자의 작업을 소비자 스레드에게 직접 전달한다.
- 쉽게 이야기해서 저장 공간의 크기가 0이고, 생산자 스레드가 큐가 작업을 전달하면 소비자 스레드가 큐에서 작업을 꺼낼 때 까지 대기한다.
- 소비자 작업을 요청하면 기다리던 생산자가 소비자에게 직접 작업을 전달하고 반환된다. 그 반대의 경우도 같다.
- 이름 그대로 생산자와 소비자를 동기화하는 큐이다.
- 쉽게 이야기해서 중간에 버퍼를 두지 않는 스레드간 직거래라고 생각하면 된다.

코드로 실행해보자.

```
package thread.executor.poolsize;

import thread.executor.RunnableTask;

import java.util.concurrent.*;

import static thread.executor.ExecutorUtils.printState;
import static util.MyLogger.log;
import static util.ThreadUtils.sleep;

public class PoolSizeMainV3 {

    public static void main(String[] args) throws InterruptedException {
        //ExecutorService es = Executors.newCachedThreadPool();
        //keepAliveTime 60초 -> 3초로 조절
        ThreadPoolExecutor es = new ThreadPoolExecutor(0, Integer.MAX_VALUE,
3, TimeUnit.SECONDS, new SynchronousQueue<>());
        log("pool 생성");
        printState(es);

        for (int i = 1; i <= 4; i++) {
            String taskName = "task" + i;
            es.execute(new RunnableTask(taskName));
            printState(es, taskName);
        }

        sleep(3000);
        log("== 작업 수행 완료 ==");
        printState(es);

        sleep(3000);
        log("== maximumPoolSize 대기 시간 초과 ==");
    }
}
```

```

        printState(es);

        es.close();
        log("== shutdown 완료 ==");
        printState(es);
    }

}

```

- 캐시 스레드 풀 전략은 초과 스레드가 60초의 생존 주기를 가지지만, 여기서는 확인을 위해 3초로 조절하겠다.

실행 결과

```

16:12:31.681 [      main] pool 생성
16:12:31.690 [      main] [pool=0, active=0, queuedTasks=0, completedTasks=0]
16:12:31.692 [pool-1-thread-1] task1 시작
16:12:31.697 [      main] task1 -> [pool=1, active=1, queuedTasks=0,
completedTasks=0]
16:12:31.697 [      main] task2 -> [pool=2, active=2, queuedTasks=0,
completedTasks=0]
16:12:31.697 [pool-1-thread-2] task2 시작
16:12:31.697 [      main] task3 -> [pool=3, active=3, queuedTasks=0,
completedTasks=0]
16:12:31.697 [pool-1-thread-3] task3 시작
16:12:31.697 [pool-1-thread-4] task4 시작
16:12:31.697 [      main] task4 -> [pool=4, active=4, queuedTasks=0,
completedTasks=0]
16:12:32.698 [pool-1-thread-1] task1 완료
16:12:32.702 [pool-1-thread-2] task2 완료
16:12:32.702 [pool-1-thread-3] task3 완료
16:12:32.702 [pool-1-thread-4] task4 완료
16:12:34.703 [      main] == 작업 수행 완료 ==
16:12:34.704 [      main] [pool=4, active=0, queuedTasks=0, completedTasks=4]
16:12:37.707 [      main] == maximumPoolSize 대기 시간 초과 ==
16:12:37.709 [      main] [pool=0, active=0, queuedTasks=0, completedTasks=4]
16:12:37.710 [      main] == shutdown 완료 ==
16:12:37.710 [      main] [pool=0, active=0, queuedTasks=0, completedTasks=4]

```

- 모든 작업이 대기하지 않고 작업의 수 만큼 스레드가 생기면서 바로 실행되는 것을 확인할 수 있다.
- "maximumPoolSize 대기 시간 초과" 로그를 통해 초과 스레드가 대기 시간이 지나서 모두 사라진 것을 확인할 수 있다.

이 전략은 다음과 같은 특징이 있다.

특징

캐시 스레드 풀 전략은 매우 빠르고, 유연한 전략이다.

이 전략은 기본 스레드도 없고, 대기 큐에 작업도 쌓이지 않는다. 대신에 작업 요청이 오면 초과 스레드로 작업을 바로바로 처리한다. 따라서 빠른 처리가 가능하다. 초과 스레드의 수도 제한이 없기 때문에 CPU, 메모리 자원만 허용한다면 시스템의 자원을 최대로 사용할 수 있다.

추가로 초과 스레드는 60초간 생존하기 때문에 작업 수에 맞추어 적절한 수의 스레드가 재사용된다. 이런 특징 때문에 요청이 갑자기 증가하면 스레드도 갑자기 증가하고, 요청이 줄어들면 스레드도 점점 줄어든다.

이 전략은 작업의 요청 수에 따라서 스레드도 증가하고 감소하므로, 매우 유연한 전략이다.

그런데 어떻게 기본 스레드 없이 초과 스레드만 만들 수 있을까? Executor 스레드 풀 기본 관리 정책을 다시 확인해보자.

Executor 스레드 풀 관리

1. 작업을 요청하면 core 사이즈 만큼 스레드를 만든다.
 - **core 사이즈가 없다. 바로 core 사이즈를 초과한다.**
2. core 사이즈를 초과하면 큐에 작업을 넣는다.
 - **큐에 작업을 넣을 수 없다.** (SynchronousQueue 는 큐의 저장 공간이 0인 특별한 큐이다.)
3. 큐를 초과하면 max 사이즈 만큼 스레드를 만든다. 임시로 사용되는 초과 스레드가 생성된다.
 - **초과 스레드가 생성된다. 물론 풀에 대기하는 초과 스레드가 있으면 재사용된다.**
4. max 사이즈를 초과하면 요청을 거절한다. 예외가 발생한다.
 - **참고로 max 사이즈가 무제한이다. 따라서 초과 스레드를 무제한으로 만들 수 있다.**

결과적으로 이 전략의 모든 작업은 초과 스레드가 처리한다.

주의

이 방식은 작업 수에 맞추어 스레드 수가 변하기 때문에, 작업의 처리 속도가 빠르고, CPU, 메모리를 매우 유연하게 사용할 수 있다는 장점이 있다. 하지만 상황에 따라서 장점이 가장 큰 단점이 되기도 한다.

상황1 - 점진적인 사용자 확대

- 개발한 서비스가 잘 되어서 사용자가 점점 늘어난다.
- 캐시 스레드 전략을 사용하면 이런 경우 크게 문제가 되지 않는다.
- 캐시 스레드 전략은 이런 경우에는 문제를 빠르게 찾을 수 있다. 사용자가 점점 증가하면서 스레드 사용량도 함께 늘어난다. 따라서 CPU 메모리의 사용량도 자연스럽게 증가한다.
- 물론 CPU, 메모리 자원은 한계가 있기 때문에 적절한 시점에 시스템을 증설해야 한다. 그렇지 않으면 CPU, 메모리 같은 시스템 자원을 너무 많이 사용하면서 시스템이 다운될 수 있다.

상황2 - 갑작스런 요청 증가

- 마케팅 팀의 이벤트가 대성공 하면서 갑자기 사용자가 폭증했다.
- 고객은 응답을 받지 못한다고 항의한다.

상황2 - 확인

- 개발자는 급하게 CPU, 메모리 사용량을 확인해보는데, CPU 사용량이 100%이고, 메모리 사용량도 지나치게 높아져있다.
- 스레드 수를 확인해보니 스레드가 수 천개 실행되고 있다. 너무 많은 스레드가 작업을 처리하면서 시스템 전체가 느려지는 현상이 발생한다.
- 캐시 스레드 풀 전략은 스레드가 무한으로 생성될 수 있다.
- 수 천개의 스레드가 처리하는 속도 보다 더 많은 작업이 들어온다.
- 시스템은 너무 많은 스레드에 잠식 당해서 거의 다운된다. 메모리도 거의 다 사용되어 버린다.
- 시스템이 멈추는 장애가 발생한다.

고정 스레드 풀 전략은 서버 자원은 여유가 있는데, 사용자만 점점 느려지는 문제가 발생할 수 있다. 반면에 캐시 스레드 풀 전략은 서버의 자원을 최대한 사용하지만, 서버가 감당할 수 있는 임계점을 넘는 순간 시스템이 다운될 수 있다.

Executor 전략 - 사용자 정의 풀 전략

상황1 - 점진적인 사용자 확대

- 개발한 서비스가 잘 되어서 사용자가 점점 늘어난다.

상황2 - 갑작스런 요청 증가

- 마케팅 팀의 이벤트가 대성공 하면서 갑자기 사용자가 폭증했다.

다음과 같이 세분화된 전략을 사용하면 상황1, 상황2를 모두 어느정도 대응할 수 있다.

- **일반:** 일반적인 상황에는 CPU, 메모리 자원을 예측할 수 있도록 고정 크기의 스레드로 서비스를 안정적으로 운영한다.
- **긴급:** 사용자의 요청이 갑자기 증가하면 긴급하게 스레드를 추가로 투입해서 작업을 빠르게 처리한다.
- **거절:** 사용자의 요청이 폭증해서 긴급 대응도 어렵다면 사용자의 요청을 거절한다.

이 방법은 평소에는 안정적으로 운영하다가, 사용자의 요청이 갑자기 증가하면 긴급하게 스레드를 더 투입해서 급한 불을 끄는 방법이다. 물론 긴급 상황에는 CPU, 메모리 자원을 더 사용하기 때문에 적정 수준을 찾아야 한다. 일반적으로는 여기까지 대응이 되겠지만, 시스템이 감당할 수 없을 정도로 사용자의 요청이 폭증하면, 처리 가능한 수준의 사용자 요청만 처리하고 나머지 요청은 거절해야 한다. 어떤 경우에도 시스템이 다운되는 최악의 상황은 피해야 한다.

세분화 전략은 다음과 같이 적용할 수 있다.

```
ExecutorService es = new ThreadPoolExecutor(100, 200, 60, TimeUnit.SECONDS,
new ArrayBlockingQueue<>(1000));
```

- 100개의 기본 스레드를 사용한다.
- 추가로 긴급 대응 가능한 긴급 스레드 100개를 사용한다. 긴급 스레드는 60초의 생존 주기를 가진다.
- 1000개의 작업이 큐에 대기할 수 있다.

코드를 실행해보자.

```
package thread.executor.poolsize;

import thread.executor.RunnableTask;

import java.util.concurrent.*;

import static thread.executor.ExecutorUtils.printState;
import static util.MyLogger.log;

public class PoolSizeMainV4 {

    static final int TASK_SIZE = 1100;    // 1. 일반
    //static final int TASK_SIZE = 1200; // 2. 긴급
    //static final int TASK_SIZE = 1201; // 3. 거절

    public static void main(String[] args) throws InterruptedException {
        ExecutorService es = new ThreadPoolExecutor(100, 200, 60,
        TimeUnit.SECONDS, new ArrayBlockingQueue<>(1000));
        printState(es);

        long startMs = System.currentTimeMillis();

        for (int i = 1; i <= TASK_SIZE; i++) {
            String taskName = "task" + i;
            try {
                es.execute(new RunnableTask(taskName));
                printState(es, taskName);
            } catch (RejectedExecutionException e) {
                log(taskName + " -> " + e);
            }
        }
    }
}
```

```

    }

    es.close();
    long endMs = System.currentTimeMillis();
    log("time: " + (endMs - startMs));
}

}

```

이 전략은 다음과 같이 작동한다.

하나의 작업을 처리하는데 약 1초가 걸린다고 가정하겠다.

주석을 변경하면 다음 시나리오를 확인할 수 있다.

- **일반:** 1000개 이하의 작업이 큐에 담겨있다. → 100개의 기본 스레드가 처리한다.
- **긴급:** 큐에 담긴 작업이 1000개를 초과한다. → 100개의 기본 스레드 + 100개의 초과 스레드가 처리한다.
- **거절:** 초과 스레드를 투입했지만, 큐에 담긴 작업 1000개를 초과하고 또 초과 스레드도 넘어간 상황이다. 이 경우 예외를 발생시킨다.

일반 - TASK_SIZE=1100

```

[    main] task1100 -> [pool=100, active=100, queuedTasks=1000,
completedTasks=0]
...
[    main] time: 11073

```

- 1000개 이하의 작업이 큐에 담겨있다. → 100개의 기본 스레드가 처리한다.
- 최대 1000개의 작업이 큐에 대기하고 100개의 작업이 실행중일 수 있다. 따라서 1100개 까지는 기본 스레드로 처리할 수 있다.
- 작업을 모두 처리하는데 11초가 걸린다. $1100 / 100 \rightarrow 11$ 초

긴급 - TASK_SIZE=1200

```

[    main] task1200 -> [pool=200, active=200, queuedTasks=1000,
completedTasks=0]
...
[    main] time: 6086

```

- 큐에 담긴 작업이 1000개를 초과한다. → 100개의 기본 스레드 + 100개의 초과 스레드가 처리한다.
- 최대 1000개의 작업이 대기하고 200개의 작업이 실행중일 수 있다.

- 작업을 모두 처리하는데 6초가 걸린다. $1200 / 200 \rightarrow 6$ 초
- 긴급 투입한 스레드 덕분에 풀의 스레드 수가 2배가 된다. 따라서 작업을 2배 빠르게 처리한다.
- 물론 CPU, 메모리 사용을 더 하기 때문에 이런 부분은 감안해서 긴급 상황에 투입할 최대 스레드를 정해야 한다.

거절 - TASK_SIZE=1201

```
[    main] task1200 -> [pool=200, active=200, queuedTasks=1000,
completedTasks=0]
[    main] task1201 -> java.util.concurrent.RejectedExecutionException: Task
thread.executor RunnableTask@7f13d6e rejected from
java.util.concurrent.ThreadPoolExecutor@7f690630[Running, pool size = 200,
active threads = 200, queued tasks = 1000, completed tasks = 0]
```

- 중간에 task1201 예외 로그를 잘 확인해야 한다.
- 긴급 투입한 스레드로도 작업이 빠르게 소모되지 않는다는 것은, 시스템이 감당하기 어려운 많은 요청이 들어오고 있다는 의미이다.
- 여기서는 큐에 대기하는 작업 1000개 + 스레드가 처리 중인 작업 200개 $\rightarrow$ 총 1200개의 작업을 초과하면 예외가 발생한다.
- 따라서 1201번에서 예외가 발생한다.
- 이런 경우 요청을 거절한다. 고객 서비스라면 시스템에 사용자가 너무 많으니 나중에 다시 시도해달라고 해야 한다.
- 나머지 1200개의 작업들은 긴급 상황과 같이 정상 처리된다.

실무에서 자주 하는 실수

참고 - 만약 다음과 같이 설정하면?

```
new ThreadPoolExecutor(100, 200, 60, TimeUnit.SECONDS, new
LinkedBlockingQueue());
```

- 기본 스레드 100개
- 최대 스레드 200개
- 큐 사이즈: 무한대

이렇게 설정하면 절대로 최대 사이즈 만큼 늘어나지 않는다. 왜냐하면 큐가 가득차야 긴급 상황으로 인지 되는데, `LinkedBlockingQueue`를 기본 생성자를 통해 무한대의 사이즈로 사용하게 되면, 큐가 가득찰 수 가 없다. 결국 기본 스레드 100개만으로 무한대의 작업을 처리해야 하는 문제가 발생한다. 실무에서 자주하는 실수 중에 하나이다.

Executor 예외 정책

생산자 소비자 문제를 실무에서 사용할 때는, 결국 소비자가 처리할 수 없을 정도로 생산 요청이 가득 차면 어떻게 할지를 정해야 한다. 개발자가 인지할 수 있게 로그도 남겨야 하고, 사용자에게 현재 시스템에 문제가 있다고 알리는 것도 필요하다.

이런 것을 위해 예외 정책이 필요하다.

`ThreadPoolExecutor`에 작업을 요청할 때, 큐도 가득차고, 초과 스레드도 더는 할당할 수 없다면 작업을 거절한다.

`ThreadPoolExecutor`는 작업을 거절하는 다양한 정책을 제공한다.

- **AbortPolicy**: 새로운 작업을 제출할 때 `RejectedExecutionException`을 발생시킨다. 기본 정책이다.
- **DiscardPolicy**: 새로운 작업을 조용히 버린다.
- **CallerRunsPolicy**: 새로운 작업을 제출한 스레드가 대신해서 직접 작업을 실행한다.
- **사용자 정의**(`RejectedExecutionHandler`): 개발자가 직접 정의한 거절 정책을 사용할 수 있다.

참고로 `ThreadPoolExecutor`를 `shutdown()` 하면 이후에 요청하는 작업을 거절하는데, 이때도 같은 정책이 적용된다.

AbortPolicy

작업이 거절되면 `RejectedExecutionException`을 던진다. 기본적으로 설정되어 있는 정책이다.

```
package thread.executor.reject;

import thread.executor.RunnableTask;

import java.util.concurrent.*;

import static util.MyLogger.log;

public class RejectMainV1 {

    public static void main(String[] args) {
        ExecutorService executor = new ThreadPoolExecutor(1, 1, 0,
            TimeUnit.SECONDS,
                new SynchronousQueue<>(), new
```

```

ThreadPoolExecutor.AbortPolicy());

    executor.submit(new RunnableTask("task1"));

    try {
        executor.submit(new RunnableTask("task2"));
    } catch (RejectedExecutionException e) {
        log("요청 초과");
        // 포기, 다시 시도 등 다양한 고민을 하면 됨
        log(e);
    }
    executor.close();
}
}

```

- ThreadPoolExecutor 생성자 마지막에 new ThreadPoolExecutor.AbortPolicy() 를 제공하면 된다.
- 참고로 이것이 기본 정책이기 때문에 생략해도 된다.
- 스레드는 1개만 사용한다. 예제를 단순하게 만들기 위해 큐에 작업을 넣지 않도록 SynchronousQueue 를 사용한다.

실행 결과

```

10:08:06.458 [      main] 요청 초과
10:08:06.458 [pool-1-thread-1] task1 시작
10:08:06.460 [      main] java.util.concurrent.RejectedExecutionException: Task
java.util.concurrent.FutureTask@79698539[Not completed, task =
java.util.concurrent.Executors$RunnableAdapter@79b4d0f[Wrapped task =
thread.executor.RunnableTask@6b2fad11]] rejected from
java.util.concurrent.ThreadPoolExecutor@2f7a2457[Running, pool size = 1,
active threads = 1, queued tasks = 0, completed tasks = 0]
10:08:07.466 [pool-1-thread-1] task1 완료

```

- task1 은 풀의 스레드가 수행한다.
- task2 를 요청하면 허용 작업을 초과한다. 따라서 RejectedExecutionException 이 발생한다.

RejectedExecutionException 예외를 잡아서 작업을 포기하거나, 사용자에게 알리거나, 다시 시도하면 된다. 이렇게 예외를 잡아서 필요한 코드를 직접 구현해도 되고, 아니면 다음에 설명한 다른 정책들을 사용해도 된다.

RejectedExecutionHandler

마지막에 전달한 `AbortPolicy`는 `RejectedExecutionHandler`의 구현체이다.
`ThreadPoolExecutor` 생성자는 `RejectedExecutionHandler`의 구현체를 전달 받는다.

```
public interface RejectedExecutionHandler {  
    void rejectedExecution(Runnable r, ThreadPoolExecutor executor);  
}
```

```
public static class AbortPolicy implements RejectedExecutionHandler {  
  
    public void rejectedExecution(Runnable r, ThreadPoolExecutor e) {  
        throw new RejectedExecutionException("Task " + r.toString() +  
            " rejected from " +  
            e.toString());  
    }  
}
```

`ThreadPoolExecutor`는 거절해야 하는 상황이 발생하면 여기에 있는 `rejectedExecution()`을 호출한다.
`AbortPolicy`는 `RejectedExecutionException`을 던지는 것을 확인할 수 있다.

DiscardPolicy

거절된 작업을 무시하고 아무런 예외도 발생시키지 않는다.

```
package thread.executor.reject;  
  
import thread.executor.RunnableTask;  
  
import java.util.concurrent.*;  
  
public class RejectMainV2 {  
  
    public static void main(String[] args) {  
        ExecutorService executor = new ThreadPoolExecutor(1, 1, 0,  
            TimeUnit.SECONDS,  
                new SynchronousQueue<>(), new  
            ThreadPoolExecutor.DiscardPolicy());  
    }  
}
```

```

        executor.submit(new RunnableTask("task1"));
        executor.submit(new RunnableTask("task2"));
        executor.submit(new RunnableTask("task3"));
        executor.close();
    }

}

```

- ThreadPoolExecutor 생성자 마지막에 new ThreadPoolExecutor.DiscardPolicy() 를 제공하면 된다.

실행 결과

```

10:08:25.414 [pool-1-thread-1] task1 시작
10:08:26.422 [pool-1-thread-1] task1 완료

```

- task2, task3 은 거절된다. DiscardPolicy 는 조용히 버리는 정책이다.
- 다음 구현 코드를 보면 왜 조용히 버리는 정책인지 이해가 될 것이다.

```

public static class DiscardPolicy implements RejectedExecutionHandler {
    public void rejectedExecution(Runnable r, ThreadPoolExecutor e) {
        // empty
    }
}

```

CallerRunsPolicy

호출한 스레드가 직접 작업을 수행하게 한다. 이로 인해 새로운 작업을 제출하는 스레드의 속도가 느려질 수 있다.

```

package thread.executor.reject;

import thread.executor.RunnableTask;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.SynchronousQueue;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;

```

```

public class RejectMainV3 {

    public static void main(String[] args) {
        ExecutorService executor = new ThreadPoolExecutor(1, 1, 0,
        TimeUnit.SECONDS,
            new SynchronousQueue<>(), new
        ThreadPoolExecutor.CallerRunsPolicy());
        executor.submit(new RunnableTask("task1"));
        executor.submit(new RunnableTask("task2"));
        executor.submit(new RunnableTask("task3"));
        executor.submit(new RunnableTask("task4"));
        executor.close();
    }
}

```

```

10:08:42.611 [pool-1-thread-1] task1 시작
10:08:42.611 [      main] task2 시작
10:08:43.617 [      main] task2 완료
10:08:43.618 [pool-1-thread-1] task1 완료
10:08:43.619 [      main] task3 시작
10:08:44.625 [      main] task3 완료
10:08:44.626 [pool-1-thread-1] task4 시작
10:08:45.629 [pool-1-thread-1] task4 완료

```

- task1 은 스레드 풀에 스레드가 있어서 수행한다.
- task2 는 스레드 풀에 보관할 큐도 없고, 작업할 스레드가 없다. 거절해야 한다.
- 이때 작업을 거절하는 대신에, 작업을 요청한 스레드에 대신 일을 시킨다.
- task2 의 작업을 main 스레드가 수행하는 것을 확인할 수 있다.

이 정책의 특징은 생산자 스레드가 소비자 대신 일을 수행하는 것도 있지만, 생산자 스레드가 대신 일을 수행하는 덕분에 작업의 생산 자체가 느려진다는 점이다. 덕분에 작업의 생산 속도가 너무 빠르다면, 생산 속도를 조절할 수 있다. 원래대로 하면 main 스레드가 task1, task2, task3, task4 를 연속해서 바로 생산해야 한다. CallerRunsPolicy 정책 덕분에 main 스레드는 task2 를 본인이 직접 완료하고 나서야 task3 을 생산할 수 있다. 결과적으로 생산 속도가 조절되었다.

```

public static class CallerRunsPolicy implements RejectedExecutionHandler {

```

```

    public void rejectedExecution(Runnable r, ThreadPoolExecutor e) {
        if (!e.isShutdown()) {
            r.run();
        }
    }
}

```

- `r.run()` 코드를 보면 별도의 스레드에서 수행하는 것이 아니라 `main` 스레드가 직접 수행하는 것을 알 수 있다.
- 참고로 `ThreadPoolExecutor` 를 `shutdown()` 을 하면 이후에 요청하는 작업을 거절하는데, 이때도 같은 정책이 적용된다.
- 그런데 `CallerRunsPolicy` 정책은 `shutdown()` 이후에도 작업을 수행해버린다. 따라서 `shutdown()` 조건을 체크해서 이 경우에는 작업을 수행하지 않도록 한다.

사용자 정의

- **사용자 정의(RejectedExecutionHandler):** 사용자는 `RejectedExecutionHandler` 인터페이스를 구현하여 자신만의 거절 처리 전략을 정의할 수 있다. 이를 통해 특정 요구사항에 맞는 작업 거절 방식을 설정할 수 있다.

```

package thread.executor.reject;

import thread.executor.RunnableTask;

import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

import static util.MyLogger.log;

public class RejectMainV4 {

    public static void main(String[] args) {
        ExecutorService executor = new ThreadPoolExecutor(1, 1, 0,
            TimeUnit.SECONDS,
            new SynchronousQueue<>(), new
            MyRejectedExecutionHandler());

        executor.submit(new RunnableTask("task1"));
        executor.submit(new RunnableTask("task2"));
    }
}

```

```

        executor.submit(new RunnableTask("task3"));
        executor.close();
    }

    static class MyRejectedExecutionHandler implements
RejectedExecutionHandler {

        static AtomicInteger count = new AtomicInteger(0);

        @Override
        public void rejectedExecution(Runnable r, ThreadPoolExecutor executor)
        {
            int i = count.incrementAndGet();
            log("[경고] 거절된 누적 작업 수: " + i);
        }
    }
}

```

- 거절된 작업을 버리지만, 대신에 경고 로그를 남겨서 개발자가 문제를 인지할 수 있도록 해보자.

```

10:09:03.646 [pool-1-thread-1] task1 시작
10:09:03.646 [      main] [경고] 거절된 누적 작업 수: 1
10:09:03.649 [      main] [경고] 거절된 누적 작업 수: 2
10:09:04.654 [pool-1-thread-1] task1 완료

```

정리

실무 전략 선택

- 고정 스레드 풀 전략: 트래픽이 일정하고, 시스템 안전성이 가장 중요
- 캐시 스레드 풀 전략: 일반적인 성장하는 서비스
- 사용자 정의 풀 전략: 다양한 상황에 대응

가장 좋은 최적화는 최적화하지 않는 것이다.

많은 개발자가 미래에 발생하지 않을 일 때문에 코드를 최적화하는 경우가 많다. 예를 들어서 초기 서비스이고, 아직 사용자가 많을지 예측이 되지 않는 상황인데, 코드 최적화에 너무 많은 시간을 사용할 수 있다. 이것은 사용자는 얼마 없는

데 매우 비싼 서버를 구매하는 것과 같다. 물론 이 이야기가 극단적으로 최적화를 하지 말자는 말이 아니다.

예를 들어서 A와 관련된 기능을 매우 많이 최적화 했는데, 사용자가 없어서 결국 버리게 되는 경우도 있다. 반면에 별로 신경쓰지 않은 B와 관련된 기능에 사용자가 많이 늘어날 수도 있다.

중요한 것은 예측 불가능한 너무 먼 미래 보다는 현재 상황에 맞는 최적화가 필요하다는 점이다.

시스템의 상황을 잘 모니터링 하고 있다가, 최적화가 필요한 부분들이 발생하면, 그때 필요한 부분들을 개선하는 것이다.

우리가 만든 서비스가 잘 되어서 많은 요청이 들어오면 좋겠지만, 대부분의 서비스는 트래픽이 어느정도 예측 가능하다. 그리고 성장하는 서비스라도 어느정도 성장이 예측 가능하다.

그래서 일반적인 상황이라면 고정 스레드 풀 전략이나, 캐시 스레드 풀 전략을 사용하면 충분하다.

한번에 처리할 수 있는 수를 제안하고 안정적으로 처리하고 싶다면 고정 풀 전략을 선택하고, 사용자의 요청을 빠르게 대응하고 싶다면 캐시 스레드 풀 전략을 사용하면 된다. 물론 자원만 충분하다면 고정 풀 전략을 선택하면서 풀의 수를 많이 늘려서 사용자의 요청도 빠르게 대응하면서 안정적인 서비스 운영도 가능하다.

그러다가 일반적인 상황을 벗어날 정도로 서비스가 잘 운영되면 그때 더 나은 최적화 방법을 선택하면 된다.

백엔드 서버 개발자라면 시스템의 자원을 적절하게 활용하되, 최악의 경우 적절한 거절을 통해 시스템이 다운되지 않도록 해야 한다.

적절한 거절은 서버도 우리의 삶에도 모두 필요하다.